The Journal of Chemical Physics

RESEARCH ARTICLE | JUNE 16 2023

Distributed memory, GPU accelerated Fock construction for hybrid, Gaussian basis density functional theory

Special Collection: High Performance Computing in Chemical Physics

David B. Williams-Young ; Andrey Asadchev; Doru Thom Popovici ; David Clark; Jonathan Waldrop ; Theresa L. Windus ; Edward F. Valeev ; Wibe A. de Jong

J. Chem. Phys. 158, 234104 (2023)
https://doi.org/10.1063/5.0151070
CHORUS

The Journal of Chemical Physics ARTICLE scitation.org/journal/jcp

Distributed memory, GPU accelerated Fock construction for hybrid, Gaussian basis density functional theory

Cite as: J. Chem. Phys. 158, 234104 (2023); doi: 10.1063/5.0151070
Submitted: 18 March 2023 • Accepted: 26 May 2023 •
Published Online: 16 June 2023

David B. Williams-Young,[1,a] Andrey Asadchev,[2] Doru Thom Popovici,[1] David Clark,[3] Jonathan Waldrop,[4] Theresa L. Windus,[4,5] Edward F. Valeev,[2,a] and Wibe A. de Jong[1]

AFFILIATIONS

[1] Applied Mathematics and Computational Research Division, Lawrence Berkeley National Laboratory, Berkeley, California 94720, USA
[2] Department of Chemistry, Virginia Tech, Blacksburg, Virginia 24061, USA
[3] NVIDIA Corporation, Santa Clara, California 95051, USA
[4] Chemical and Biological Sciences Division, Ames National Laboratory, Ames, Iowa 50011, USA
[5] Department of Chemistry, Iowa State University, Ames, Iowa 50011, USA

Note: This paper is part of the JCP Special Topic on High Performance Computing in Chemical Physics.
[a] **Authors to whom correspondence should be addressed:** dbwy@lbl.gov and efv@vt.edu

ABSTRACT

With the growing reliance of modern supercomputers on accelerator-based architecture such a graphics processing units (GPUs), the development and optimization of electronic structure methods to exploit these massively parallel resources has become a recent priority. While significant strides have been made in the development GPU accelerated, distributed memory algorithms for many modern electronic structure methods, the primary focus of GPU development for Gaussian basis atomic orbital methods has been for shared memory systems with only a handful of examples pursing massive parallelism. In the present work, we present a set of distributed memory algorithms for the evaluation of the Coulomb and exact exchange matrices for hybrid Kohn–Sham DFT with Gaussian basis sets via direct density-fitted (DF-J-Engine) and seminumerical (sn-K) methods, respectively. The absolute performance and strong scalability of the developed methods are demonstrated on systems ranging from a few hundred to over one thousand atoms using up to 128 NVIDIA A100 GPUs on the Perlmutter supercomputer.

I. INTRODUCTION

Since its inception, quantum chemistry has relied on its ability to quickly adapt to an ever-evolving landscape of computer architectures to enable the next generation of scientific applications. A particular emphasis has been placed on the leverage of distributed memory parallelism to enable *ab initio* simulations of large molecular systems on the world's largest supercomputers.[1–4] The past two decades have been no different, with an enormous research effort having been afforded to targeting the latest introduction into the high-performance computing (HPC) ecosystem: graphics processing units (GPUs).[4–6] The use of GPUs in quantum chemistry is relatively long standing, with applications ranging from single-body methods, such as Hartree–Fock (HF) and density functional (DFT) theories,[7–39] to many-body methods,[3] such as coupled-cluster theory,[40–49] many-body perturbation theory,[50–56] and configuration interaction and complete active space theories[57–61] to name a few. Despite significant advances in the development of GPU accelerated quantum chemistry software, these efforts are not yet as mature as their central processing unit (CPU) counterparts, and growing requirements for the desired scale and accuracy of *ab initio* simulations require further development in this area. As such, the pursuance of improved, GPU accelerated quantum chemistry methods capable of leveraging the latest advances in modern HPC remains an active area of research.

With the growing reliance of modern exascale HPC systems on accelerator architectures,[62,63] recent years have seen the deployment of GPUs in distributed memory systems. The memory model of these systems is hierarchical, with each GPU device having its own set of high-bandwidth memory and, in most contemporary HPC systems, several GPU devices are connected to a CPU host node that in turn has its own local memory disjoint from its connected devices. Within a compute node, a significant optimization challenge is in the movement of data between host and device memory spaces either via explicit data copies or, more recently, unified memory models. In distributed memory systems, individual compute nodes are connected via comparatively slow interconnect, which is accompanied by its own set of unique optimization challenges. In particular, the increased local processing rate of GPU accelerated compute nodes has exposed bottlenecks involving communication and load imbalance that were less apparent on the massively parallel CPU systems of years past.[63–65]

As necessitated by the predominant availability of consumer-grade, gaming-oriented GPU hardware, early development of GPU accelerated quantum chemistry methods focused on shared memory and single-node systems where host and device memory spaces, while disjoint, are accessible to one another without the need for communication across distributed memory networks. Due to their relative computational cost, many-body methods (e.g., coupled-cluster theory[46–49]) and spectral single-body methods (e.g., planewave,[27–32] real-space,[33,34] finite-element,[35] and wavelet[36] discretizations of DFT) were among the first to be successfully ported to distributed memory GPU architectures. The success of these developments has been, in large part, enabled by the prevalent use of GPU optimized libraries for common operations, such as (multi-)linear algebra[43,66–69] and fast Fourier transforms (FFT),[70–72] in these applications. On the other hand, atomic orbital (AO) single-body methods, such as Gaussian, Slater, and numerical atomic orbital HF and DFT, present a unique challenge for GPU optimization due to their significant dependence on domain-specific kernels (e.g., AO integrals and highly specialized numerical integration techniques) that exhibit less-regular compute patterns than those common operations aforementioned. For Gaussian basis DFT in particular, kernels involving manipulation of the AO electron-repulsion integral (ERI) tensor to form the Coulomb and exchange matrices are among the most cumbersome for GPU architectures. As such, the vast majority of both shared[7,8,10,13,16,20,21,25,73–76] and distributed[11,12,24,77,78] memory Gaussian DFT GPU efforts has centered around optimization of these terms.

The primary innovations of the present work concern the evaluation of Coulomb and exact exchange matrices. Although the naive (based on 2-body 4-center ERI) approach for evaluation of these contributions is simple and thus used by majority of implementations on distributed heterogeneous platforms,[11,12,23] their steep asymptotic scaling complicates their application to large molecular systems. For the Coulomb matrix, the use of 4-center integrals results in suboptimal $\mathcal{O}(N^2)$ cost, whereas fast approaches with $\mathcal{O}(N)$ for the Coulomb potential (CP) evaluation are well known.[79,80] More importantly, even if combined with the fast $\mathcal{O}(N)$ treatment of the long-range contributions to the potential, the use of 4-center integrals for the near-field contributions is still prohibitively expensive. The solution is well known also: Clever optimizations (like early integral digestion[81–84]) and/or numerical approximations like density fitting (DF)[85,86] can dramatically reduce the cost of Coulomb matrix evaluation such that even with naive $\mathcal{O}(N^2)$ evaluation, its cost is dwarfed by the cost of the exact exchange.

While with proper screening the exact exchange evaluation is $\mathcal{O}(N)$ in system size, the use of 4-center integrals, again, results in suboptimally high prefactor. Thus, there has been a recent resurgence of interest in the development of numerical methods for the evaluation of exact exchange for molecular systems. In these methods, one of the two ERI coordinate integrations is replaced by a numerical integration. The general concept for this approach has been around for over 30 years, beginning with the pseudospectral method of Friesner and co-workers[87,88] in the early 1980s. This class of techniques was revisited in the early 2000s as the chain-of-spheres exchange (COSX) method reported in the work of Neese *et al.*[89] and more recently as the seminumerical exchange (sn-K) method reported in the work of Laqua *et al.*[10] The primary differences between these approaches are in their consideration of numerical sparsity and spatial locality and in their attempts to reduce the required size of the numerical quadrature to attain a particular accuracy. We will adopt the sn-K nomenclature in the following. The sn-K algorithm is particularly attractive for GPU hardware as there is a natural expression of vectorization in the numerical quadrature that is superior to that of analytical integral methods. Significant progress has been made in the development of GPU sn-K algorithms for shared memory systems including multiple GPUs per node.[10,13,14,90] These methods have demonstrated a sizable performance improvement over analytical integral methods on GPU architectures but have yet to be explored in the context of distributed memory parallelism.

In addition to the Coulomb and exact exchange terms, hybrid AO DFT methods also require the evaluation of the exchange–correlation (XC) potential matrix by numerical integration methods due to the nonlinear nature of the XC energy functional. In contrast to AO ERI methods, numerical integration methods developed for molecular DFT are much simpler to port to GPU architectures[7,17] and are able to heavily utilize optimized basic linear algebra subroutines (BLAS) libraries to attain near peak floating point performance on modern GPU hardware.[18] Recently, there have been a number of works addressing key optimization challenges pertaining to the distributed memory evaluation of the XC potential on GPU accelerated computing clusters.[17,24] In this work, we extend the parallel integration infrastructure developed for the XC potential by the authors[17] to treat the Gaussian basis sn-K method on distributed memory architectures.

The remainder of this work is organized as follows. In Sec. II, we review the salient aspects of Gaussian basis DFT necessary to develop parallel algorithms for the evaluation of the Coulomb (Sec. II B) and exact exchange (Sec. II D) matrices. In Secs. II E and II F, we described the extension of the distributed memory integration procedure developed for the XC potential (Sec. II C) to treat sn-K on GPU clusters. In Sec. III, we demonstrate the strong scaling performance of the developed methods for a range of systems and, finally, discuss future outlook for further development of distributed memory GPU algorithms based on the methods presented in this work in Sec. IV.

II. THEORY AND IMPLEMENTATION

A. The hybrid Kohn–Sham Fock matrix

Given a set of basis functions, $\mathcal{B} = \{\phi_\mu : \mathbb{R}^3 \to \mathbb{R}\}_{\mu=1}^{N_b}$, the hybrid Kohn–Sham (KS) Fock matrix, $\mathbf{F} \in \mathbb{R}^{N_b \times N_b}$, is given by[91,92]

$$\mathbf{F} = \mathbf{h} + \mathbf{J}[\mathbf{D}] - c_x \mathbf{K}[\mathbf{D}] + \mathbf{V}^{xc}[\mathbf{D}], \tag{1}$$

where $\mathbf{h}$ is the core Hamiltonian containing the free-particle (kinetic) Hamiltonian and the electron–nuclear interaction potential, and $\mathbf{D}$ is the basis-discretized one-particle density matrix. $\mathbf{J}$, $\mathbf{K}$, and $\mathbf{V}^{xc}$ are the density-dependent (as denoted by $[\,\cdot\,]$) Coulomb, exact exchange, and exchange–correlation (XC) potential matrices, respectively, which describe the mean-field electron–electron interactions in the hybrid KS model as modulated by the exact exchange parameter $c_x \in \mathbb{R}^+$. For real-valued basis functions and density matrices, these matrices take the forms[92]

$$J_{\mu\nu} = \sum_{\lambda\kappa} (\mu\nu|\lambda\kappa) D_{\lambda\kappa}, \tag{2}$$

$$K_{\mu\nu} = \sum_{\lambda\kappa} (\mu\lambda|\nu\kappa) D_{\lambda\kappa}, \tag{3}$$

$$V^{xc}_{\mu\nu} = \int_{\mathbb{R}^3} \mathrm{d}^3\mathbf{r}\, \phi_\mu(\mathbf{r}) \frac{\delta E^{xc}[\rho(\mathbf{r})]}{\delta\rho(\mathbf{r})} \phi_\nu(\mathbf{r}), \tag{4}$$

where E^{xc} is the XC energy functional evaluated at the electron density, $\rho : \mathbb{R}^3 \to \mathbb{R}$,

$$\rho(\mathbf{r}) = \sum_{\mu\nu} D_{\mu\nu} \phi_\mu(\mathbf{r}) \phi_\nu(\mathbf{r}) \tag{5}$$

and

$$(\mu\lambda|\nu\kappa) = \iint_{\mathbb{R}^3} \mathrm{d}^3\mathbf{r}\, \mathrm{d}^3\mathbf{r}' \frac{\phi_\mu(\mathbf{r})\phi_\lambda(\mathbf{r})\phi_\nu(\mathbf{r}')\phi_\kappa(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|} \tag{6}$$

is the electron–repulsion integral (ERI) tensor.

In this work, we take $\mathcal{B}$ to be comprised of contracted, atom-centered Gaussian-type orbitals (GTOs); however, we note that the general principles presented here may be extended to other atom-centered Ansätze, such as Slater-type orbitals (STOs), as well. We denote the atomic center of ϕ_μ as $\mathbf{R}_\mu$ in the following. GTO Fock matrix construction is dominated by the three density-dependent terms, which must be evaluated for each new density in, e.g., a self-consistent field (SCF) optimization or dynamics simulation. In the remainder of this section, we examine the scalable evaluation of these terms on distributed memory GPU architectures.

B. The Coulomb matrix via density-fitting J-engine

Efficient evaluation of the Coulomb potential given by Eq. (2) takes advantage of the density fitting (DF; also known as the Resolution-of-the-Identity) approximation of the two-electron integrals,

$$(\mu\nu|\lambda\kappa) \approx \sum_{KL} (\mu\nu|K)(\mathbf{V}^{-1})_{KL}(L|\lambda\kappa) \tag{7}$$

where 3- and 2-center Coulomb integrals involving the (auxiliary) density-fitting basis $\{\chi_K : \mathbb{R}^3 \to \mathbb{R}\}_{K=1}^{N_{\text{aux}}}$ were introduced,

$$(\mu\nu|K) = \iint_{\mathbb{R}^3} \mathrm{d}^3\mathbf{r}\mathrm{d}^3\mathbf{r}' \frac{\phi_\mu(\mathbf{r})\phi_\lambda(\mathbf{r})\chi_K(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|}, \tag{8}$$

$$V_{LK} = \iint_{\mathbb{R}^3} \mathrm{d}^3\mathbf{r}\mathrm{d}^3\mathbf{r}' \frac{\chi_L(\mathbf{r})\chi_K(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|}. \tag{9}$$

DF approximation of Eq. (2) leads to the following factorization of the Coulomb potential:

$$V_L = \sum_{\lambda\kappa} (L|\lambda\kappa) D_{\lambda\kappa}, \tag{10}$$

$$D_K = \sum_L (\mathbf{V}^{-1})_{KL} V_L, \tag{11}$$

$$J_{\mu\nu} \overset{\text{DF}}{\approx} \sum_K (\mu\nu|K) D_K, \tag{12}$$

with V_L and D_K representing the Coulomb potential of the exact density ρ and the *robust* approximation of ρ in the DF basis, respectively. DF-based evaluation of J costs $\mathcal{O}(N^3)$ in the traditional approach where the Cholesky factorization of the positive-definite matrix $\mathbf{V}$ is computed and stored (this allows us to amortize its cost over the SCF iterations). In practice, however, the cost is largely controlled by the evaluation of $\mathcal{O}(N^2)$ non-negligible Coulomb 3-center AO integrals in Eqs. (10) and (12). While for 3-center AO integrals dedicated optimization of AO integral evaluation is possible,[93–95] including specific developments for the GPU architectures,[96] optimal evaluation of Eqs. (10) and (12) involves blurring the line between integral evaluation and density contraction via several related ideas[81–84] that is often dubbed the *J-engine* approach. The original J-engine approaches were demonstrated in the context of Eq. (2); in the work of Kussmann *et al.*,[13] the utility of the McMurchie–Davidson-based[97] J-engine[84] in the DF context has been recently illustrated. Here, we only briefly recap the DF-based J-engine ("DF-J-engine") formalism using the established notation[98] for Gaussian AO integrals.

An uncontracted primitive Cartesian Gaussian with exponent $\zeta_a \in \mathbb{R}^+$ and non-negative integer Cartesian "quanta" $\mathbf{a} \equiv \{a_x, a_y, a_z\}$ centered at $\mathbf{R}_a \equiv \{A_x, A_y, A_z\}$ will be denoted by

$$\phi_{\mathbf{a}}(\mathbf{r}) \equiv x_A^{a_x} y_A^{a_y} z_A^{a_z} \exp(-\zeta_a r_A^2), \tag{13}$$

with $\mathbf{r}_A \equiv \{x_A, y_A, z_A\}, x_A \equiv x - A_x$, etc., $l_{\mathbf{a}} \equiv a_x + a_y + a_z \geq 0$ is colloquially referred to as the "angular momentum" of a Gaussian. Closely related to the Cartesian Gaussian is a Hermite Gaussian,

$$\Lambda_{\tilde{\mathbf{a}}}(\mathbf{r}) \equiv \left(\frac{\partial}{\partial x_A}\right)^{\tilde{a}_x} \left(\frac{\partial}{\partial y_A}\right)^{\tilde{a}_y} \left(\frac{\partial}{\partial z_A}\right)^{\tilde{a}_z} \exp(-\zeta_a r_A^2). \tag{14}$$

A primitive Cartesian Gaussian and a product of two primitive Cartesian Gaussians can be expressed as linear combinations of Hermite Gaussians,

$$\phi_{\mathbf{a}}(\mathbf{r}) = \sum_{\tilde{a}_x=0}^{\tilde{a}_x \leq a_x} E_{a_x}^{\tilde{a}_x} \sum_{\tilde{a}_y=0}^{\tilde{a}_y \leq a_y} E_{a_y}^{\tilde{a}_y} \sum_{\tilde{a}_z=0}^{\tilde{a}_z \leq a_z} E_{a_z}^{\tilde{a}_z} \Lambda_{\tilde{\mathbf{a}}}(\mathbf{r}) \equiv \sum_{\tilde{\mathbf{a}}} E_{\mathbf{a}}^{\tilde{\mathbf{a}}} \Lambda_{\tilde{\mathbf{a}}}(\mathbf{r}), \tag{15}$$

$$\phi_{\mathbf{a}}(\mathbf{r})\phi_{\mathbf{b}}(\mathbf{r}) = \sum_{\tilde{p}_x=0}^{\tilde{p}_x \le a_x+b_x} (E_x)_{a_x b_x}^{\tilde{p}_x} \sum_{\tilde{p}_y=0}^{\tilde{p}_y \le a_y+b_y} (E_y)_{a_y b_y}^{\tilde{p}_y} \times \sum_{\tilde{p}_z=0}^{\tilde{p}_z \le a_z+b_z} (E_z)_{a_z b_z}^{\tilde{p}_z} \Lambda_{\tilde{\mathbf{p}}}(\mathbf{r}) \equiv \sum_{\tilde{\mathbf{p}}} E_{\mathbf{ab}}^{\tilde{\mathbf{p}}} \Lambda_{\tilde{\mathbf{p}}}(\mathbf{r}), \quad (16)$$

with $\zeta_p \equiv \zeta_a + \zeta_b$, $\mathbf{R}_p \equiv \frac{\zeta_a \mathbf{R}_a + \zeta_b \mathbf{R}_b}{\zeta_a + \zeta_b}$, and $E_{\mathbf{a}}^{\tilde{\mathbf{a}}} \equiv \prod_{i=x,y,z} E_{a_i}^{\tilde{a}_i}$, $E_{\mathbf{ab}}^{\tilde{\mathbf{p}}} \equiv \prod_{i=x,y,z} (E_i)_{a_i b_i}^{\tilde{p}_i}$. The Hermite-to-Cartesian transformation coefficients are evaluated straightforwardly by recursion.[97]

The use of Eqs. (15) and (16) allows us to express a 3-center Coulomb integral over primitive Cartesian Gaussians as a linear combination,

$$(\mathbf{ab}|\mathbf{c}) = \sum_{\tilde{\mathbf{p}},\tilde{\mathbf{c}}} E_{\mathbf{ab}}^{\tilde{\mathbf{p}}} E_{\mathbf{c}}^{\tilde{\mathbf{c}}} (\tilde{\mathbf{p}}|\tilde{\mathbf{c}}), \quad (17)$$

of the Coulomb integral between two primitive Hermite Gaussians:

$$(\tilde{\mathbf{p}}|\tilde{\mathbf{c}}) \equiv \iint_{\mathbb{R}^3} \mathrm{d}^3\mathbf{r}\mathrm{d}^3\mathbf{r}' \frac{\Lambda_{\tilde{\mathbf{p}}}(\mathbf{r})\Lambda_{\tilde{\mathbf{c}}}(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|}. \quad (18)$$

The latter can be evaluated directly,

$$(\tilde{\mathbf{p}}|\tilde{\mathbf{c}}) \equiv (-1)^{l_{\tilde{\mathbf{c}}}} (\tilde{\mathbf{p}} + \tilde{\mathbf{c}})^{(0)}, \quad (19)$$

from the auxiliary integral,

$$(\tilde{\mathbf{r}})^{(m)} \equiv \left(\frac{\partial}{\partial x_R}\right)^{\tilde{r}_x} \left(\frac{\partial}{\partial y_R}\right)^{\tilde{r}_y} \left(\frac{\partial}{\partial z_R}\right)^{\tilde{r}_z} (\mathbf{0})^{(m)}, \quad (20)$$

with $(\mathbf{0})^{(m)}$ related to the Boys function $F_m(x)$,

$$(\mathbf{0})^{(m)} \equiv (-2\rho)^m \frac{2\pi^{5/2}}{\zeta_p\zeta_c\sqrt{\zeta_p+\zeta_c}} F_m(\rho|\mathbf{R}_p - \mathbf{R}_c|^2), \quad (21)$$

$$F_m(x) \equiv \int_0^1 \mathrm{d}y\, y^{2m} \exp(-xy^2), \quad (22)$$

$$\rho \equiv \frac{\zeta_p\zeta_c}{\zeta_p+\zeta_c}. \quad (23)$$

Auxiliary integrals are evaluated recursively,

$$(\tilde{\mathbf{r}} + \mathbf{1}_i)^{(m)} = \tilde{r}_i(\tilde{\mathbf{r}} - \mathbf{1}_i)^{(m+1)} + (P_i - C_i)(\tilde{\mathbf{r}})^{(m+1)}, \quad (24)$$

starting from $(\mathbf{0})^{(m)}$.

Efficient evaluation of Eqs. (10) and (12) involves contracting densities with Hermite-to-Cartesian transformation coefficients of the ket functions [Eqs. (15) and (16), respectively] to produce "Hermite" densities that can be stored and reused for every bra function in Eqs. (10) and (12), as illustrated here for a single primitive shell contribution to Eq. (10),

$$\sum_{\mathbf{ab}} (\mathbf{c}|\mathbf{ab}) D_{\mathbf{ab}} \overset{\text{Eq. (17)}}{=} \sum_{\mathbf{ab}} \left(\sum_{\tilde{\mathbf{c}}\tilde{\mathbf{p}}} E_{\mathbf{c}}^{\tilde{\mathbf{c}}} (\tilde{\mathbf{c}}|\tilde{\mathbf{p}}) E_{\mathbf{ab}}^{\tilde{\mathbf{p}}} \right) D_{\mathbf{ab}} \quad (25)$$

$$= \sum_{\tilde{\mathbf{c}}} E_{\mathbf{c}}^{\tilde{\mathbf{c}}} \left(\sum_{\tilde{\mathbf{p}}} (\tilde{\mathbf{c}}|\tilde{\mathbf{p}}) D_{\tilde{\mathbf{p}}} \right), \quad (26)$$

where

$$D_{\tilde{\mathbf{p}}} \equiv \sum_{\mathbf{ab}} E_{\mathbf{ab}}^{\tilde{\mathbf{p}}} D_{\mathbf{ab}} \quad (27)$$

and the order of evaluation is indicated by the parentheses. Refactorization of Eq. (25) via Eq. (26) is the key idea of J-engine—it leads to great FLOP reduction, which is easily rationalized as follows: Instead of multiplying "matrix" $(\tilde{\mathbf{c}}|\tilde{\mathbf{p}})$ by "matrix" $E_{\mathbf{ab}}^{\tilde{\mathbf{p}}}$, then evaluating the inner product with "vector" $D_{\mathbf{ab}}$, in the J-engine factorization the inner product of "matrix" $(\tilde{\mathbf{c}}|\tilde{\mathbf{p}})$ with "vector" $D_{\tilde{\mathbf{p}}}$ is evaluated directly. This can also be viewed as early "digestion" of the integrals; more general framework for early digestion of the integrals beyond the J matrix evaluation has been considered in the work of Adams *et al.*[83]

Equations (10) and (12) of the DF-J-engine approach were implemented in the open-source `LibintX` library[96,99] and Eq. (11) was implemented using the TiledArray library for distributed tensor contractions.[66,100] Evaluation of Eq. (10) in `LibintX` proceeds as follows:

1. **CPU**: construct a batch of **ab** shell pairs with same $l_{\text{ket}} = l_{\mathbf{a}} + l_{\mathbf{b}}$ such that (1) $D_{\mathbf{ab}}$ is local, (2) not all integrals $(\mathbf{c}|\mathbf{ab})$ are negligible, and (3) the estimated amount of device memory does not exceed that of the preallocated buffer (can be controlled by the user); pack the $D_{\mathbf{ab}}$ batch to the mapped host memory buffer and copy the packed batch data to the preallocated device data buffer;
2. **GPU**: launch a kernel to transform current batch of $D_{\mathbf{ab}}$ to $D_{\tilde{\mathbf{p}}}$ [Eq. (27)];
3. **GPU**: launch kernels to evaluate Hermite integrals [Eq. (19)] and their contributions to the Coulomb potential [Eq. (26)] for all shells **c** (one kernel per $l_{\mathbf{c}}$);
4. repeat for the next **ab** batch.

Note after launching the GPU kernels, the CPU starts to work on preparing the density and metadata for the next batch, thus the GPU and CPU work overlaps. Copying of the $D_{\mathbf{ab}}$ density batches to the GPU is synchronous, but since each batch is used in combination with many $(\mathbf{c}|$ bras in Eq. (25), the cost of the data transfers is negligible. Note that no significant metadata is kept persistently between individual J matrix builds to allow the use of full GPU memory by other application stages.

Evaluation of Eq. (12) in `LibintX` is only slightly more complicated:

1. **CPU**: construct a batch of non-negligible **ab** shell pairs with same $l_{\text{bra}} = l_{\mathbf{a}} + l_{\mathbf{b}}$ such that $J_{\mathbf{ab}}$ is local (batch size is controlled by the size of the preallocated memory buffer);
2. **GPU**: launch a kernel to evaluate $E_{\mathbf{ab}}^{\tilde{\mathbf{p}}}$;
3. **GPU**: launch kernels to evaluate Hermite density $D_{\tilde{\mathbf{c}}} \equiv \sum_{\mathbf{c}} E_{\mathbf{c}}^{\tilde{\mathbf{c}}} D_{\mathbf{c}}$, Hermite integrals [Eq. (19)], and their contributions to the Coulomb potential in the Hermite basis,

$$J_{\tilde{\mathbf{p}}} = \sum_{\tilde{\mathbf{c}}} (\tilde{\mathbf{p}}|\tilde{\mathbf{c}}) D_{\tilde{\mathbf{c}}}, \quad (28)$$

[this is analogous to the first step in Eq. (26), but adapted to the evaluation of Eq. (12)] for all shells **c** (one kernel per $l_{\mathbf{c}}$);

4. **GPU**: launch kernel to transform $J_{\bar{\mathbf{p}}}$ to $J_{\mathbf{ab}}$ via

$$J_{\mathbf{ab}} = \sum_{\bar{\mathbf{p}}} E^{\bar{\mathbf{p}}}_{\mathbf{ab}} J_{\bar{\mathbf{p}}}; \tag{29}$$

5. **CPU**: store $J_{\mathbf{ab}}$ shell-sets;
6. repeat for the next **ab** batch.
   CPU and GPU work is again overlapped, with the CPU thread starting to work on steps 1–3 of the next batch while the work on the previous batch is being completed; step 4 is scheduled only after completion of the previous batch's step 5 so that the J matrix contributions from the previous batch are not overwritten.

The work distribution among the message passing interface (MPI) ranks in `LibintX`'s DF-J-engine is statically determined by the distribution of the user-provided density matrix among ranks and by the expected distribution of the J matrix result. This design decision is motivated by the desire to simplify the application programing interface (API) of the `LibintX` library and avoid mandating the density to be in any particular data structure or distribution manner; the user provides the density to the library in the form of a C++ lambda function (closure) that provides one block of the density matrix at a time (or fails to provide it if it is not located on the current node). Production computations in Sec. III provided density and stored the resulting J matrix as a block-sparse distributed array (`DistArray`) object of the `TiledArray` framework.[67,100] Tiling of the density matrix was determined by the k-means-based clustering of the atoms,[101] with tiles divided evenly among ranks (for p distributed ranks, the first n^2/p tile ordinals assigned to rank 0, the next n^2/p tile ordinals assigned to rank 1, etc.; note that due to sparsity actual tile counts per rank are lower and end up being problem dependent). In evaluation of Eq. (10), each rank evaluates all contributions that involve the tiles of the density matrix that are screened out and that reside on that rank; each rank produces contributions to all elements of vector V_L, thus these contributions are reduced across all ranks before evaluating Eq. (11). A similar lambda-based mechanism is used by `LibintX` to return the distributed J matrix to the user and distribute the work in evaluation of Eq. (12).

In the interest of keeping the focus on the distributed memory parallelization, the kernel-level implementation details of `LibintX`'s DF-J-engine will be kept to a minimum here. Each GPU thread evaluates the entire set of "1-center" auxiliary integrals [Eq. (24)], converts them to "2-center" integrals via Eq. (19), and "digests" them via Eq. (26) [or its analog for Eq. (12)] for a single primitive shell. This allows the algorithm to completely eliminate thread divergences since every thread in a thread block performs exactly the same computation. This approach is radically different from the work distribution when computing integrals[96] in which multiple threads cooperatively evaluated shell components; this is possible due to the greatly reduced memory requirements of the DF-J-engine compared to the traditional (AO integral based) implementation of DF-J. A key feature of the kernel implementation is the use of compile-time code generation (class/function templates) assisted by compile-time function evaluation available in recent C++/CUDA standards; by offloading code generation and optimization to the C++ compiler simplifies library engineering and use by eliminating the need for a separate optimizing compiler (code generator). Such design choice is in the same vein as our recent implementation of the 3-center AO integral evaluation (rather than the J-engine).[96]

C. The exchange–correlation potential matrix

Unlike **J**, which may be assembled directly from analytical integrals, the integrals involved in $\mathbf{V}^{xc}$ [Eq. (4)] must be evaluated numerically due to the nonlinear nature of E^{xc} and its functional derivatives. As detailed elsewhere,[7,102–104] for atom-centered bases, $\mathbf{V}^{xc}$ may be efficiently evaluated via

$$V^{xc}_{\mu\nu} \approx \sum_{i \in \mathcal{Q}} \Phi_{\mu i} Z_{\nu i} + Z_{\mu i} \Phi_{\nu i}, \tag{30}$$

$$\Phi_{\mu i} = \phi_\mu(\mathbf{r}_i), \tag{31}$$

where $\mathbf{\Phi}$ is the collocation matrix and $\mathcal{Q} = \{(w_i, \mathbf{r}_i)\}_{i=1}^{N_g}$ is a numerical quadrature consisting of nodes, $\mathbf{r}_i \in \mathbb{R}^3$, and grid weights, $w_i \in \mathbb{R}$. For molecular calculations, due to the irregular nature of integrands in the vicinity of nuclear charges, $\mathcal{Q}$ is typically taken as the union of spherical quadratures $(\mathbb{R} \times S^2)$ with origin at each atomic center coupled with a modified weight partitioning scheme to account for overlapping regions. We refer the reader to Refs. 105–113 for more comprehensive discussions regarding the construction of generic molecular quadratures. In this work, atomic grids are constructed according to the Mura–Knowles (MK) scheme[108] with Lebedev–Laikov[114] angular grids, and we use the molecular weight partitioning scheme reported in the work of Stratmann *et al.*[106] In addition, angular grids are radially pruned according to the scheme of Treutler and Ahlrichs.[110]

The auxiliary matrix $\mathbf{Z} \in \mathbb{R}^{N_b \times N_g}$ is method-dependent, the form of which depends on E^{xc}. Within the spin-restricted generalized-gradient approximation (GGA), **Z** takes the form[102,103]

$$Z_{\mu i} = \frac{w_i}{2}\left(E^{\rho}_i \Phi_{\mu i} + 4E^{\gamma}_i (\nabla\rho(\mathbf{r}_i) \cdot \nabla\Phi_{\mu i})\right), \tag{32}$$

$$E^{\rho/\gamma}_i = \left.\frac{\partial \varepsilon(\rho(\mathbf{r}), \gamma(\mathbf{r}))}{\partial \rho/\gamma}\right|_{\mathbf{r}=\mathbf{r}_i}, \tag{33}$$

$$\rho(\mathbf{r}_i) = \sum_\mu F_{\mu i} \Phi_{\mu i}, \qquad \nabla\rho(\mathbf{r}_i) = 2\sum_\mu F_{\mu i} \nabla\Phi_{\mu i}, \tag{34}$$

$$F_{\mu i} = \sum_\nu D_{\mu\nu} \Phi_{\nu i}, \tag{35}$$

where $\varepsilon : \mathbb{R}^2 \to \mathbb{R}$ is the GGA energy density, which depends on ρ and $\gamma = \nabla\rho \cdot \nabla\rho$. Similar expressions have been derived for spin-generalized and meta-GGA XC functionals.[13,102,104,115] This evaluation scheme for $\mathbf{V}^{xc}$ is particularly attractive for GPU architectures due to the fact that the most compute-intensive operations can be implemented using level-3 [Eq. (30) via SYR2K and Eq. (35) via general matrix multiply (GEMM)] and level-1 [Eq. (34) via a dot product (DOT)[116]] BLAS operations.[7,13,17] As such, these operations can leverage heavily optimized GPU BLAS libraries, leaving only a small number of DFT-specific kernels [e.g., Eqs. (31)–(33)] to be optimized by the chemistry-domain developer. We will examine specific details regarding the use of GPU BLAS libraries for this application in Sec. II F.

D. The seminumerical exact exchange matrix

Although $\mathbf{K}$ can be evaluated directly from Coulomb integrals as in Eq. (3), this is rarely the most efficient approach due to the steep rise of computational cost of the 4-center integrals with the angular momenta. As detailed elsewhere,[10,87–89] the ERI tensor may be factorized on a quadrature grid via

$$(\mu\lambda|\nu\kappa) \approx \frac{1}{2}\sum_{i\in\mathcal{Q}} w_i A_{\mu\lambda i}\Phi_{\nu i}\Phi_{\kappa i} + (\mu\lambda \leftrightarrow \nu\kappa), \tag{36}$$

$$A_{\mu\lambda i} = \int_{\mathbb{R}^3} \mathrm{d}^3\mathbf{r}\, \frac{\phi_\mu(\mathbf{r})\phi_\lambda(\mathbf{r})}{|\mathbf{r}-\mathbf{r}_i|}, \tag{37}$$

where $\mathbf{A}$ is the 3-center Coulomb potential (3c-CP) integral tensor, and $\mathbf{\Phi}$ and $\mathcal{Q}$ are the collocation matrix and quadrature discussed in Sec. II C. Inserting Eq. (36) into Eq. (3), we obtain a simple expansion for $\mathbf{K}$,[10,87–89]

$$G_{\mu i} = \sum_\lambda w_i A_{\mu\lambda i} F_{\lambda i}, \tag{38}$$

$$\tilde{K}_{\mu\nu} = \sum_i G_{\mu i}\Phi_{\nu i}, \tag{39}$$

$$K_{\mu\nu} \approx \frac{1}{2}\left(\tilde{K}_{\mu\nu} + \tilde{K}_{\nu\mu}\right), \tag{40}$$

where $\mathbf{F}$ is same intermediate given in Eq. (35) for the evaluation of $\mathbf{V}^{xc}$. This method for $\mathbf{K}$ assembly will be referred to as seminumerical exchange[10] (sn-K) in the following. Apart from the use of common intermediates, there exists a striking similarity between Eqs. (38)–(40) and the $\mathbf{V}^{xc}$ assembly in Eqs. (30), (32), (34), and (35). As has been posited elsewhere,[10,89] sn-K can reuse many of the same algorithmic primitives as those used for the evaluation of $\mathbf{V}^{xc}$. One might even be tempted to implement Eq. (38) via BLAS, but we (as have others[10,89]) have found this to be inefficient in practice, particularly on GPU architectures. We present the scheme by which we perform the combined evaluation of Eqs. (37) and (38) in the Appendix.

E. Batching and screening of numerical integrals

Each of the expressions in Eqs. (30)–(35), (38), and (39) is *local* with respect to the numerical quadrature and thus may be partitioned along their grid indices (i) with final results [e.g., Eqs. (30) and (39)] being recovered as a sum over locally integrated intermediates. This partitioning has clear ramifications in the context of parallelism, which will be discussed in Sec. II F. Grid point locality also allows for the exploitation of the spatial sparsity of $\mathcal{B}$ and various operators [e.g., Eq. (37)], which in turn takes the nominal $\mathcal{O}(N_b^2 N_g)$ scaling of these numerical schemes to methods that scale (near-)linearly with respect to system size.[10,89,103,106,117,118]

As the sparsity profiles of spatially adjacent grid points are similar, it is canonical to *batch* these grid points into subsets $\mathcal{Q}_j$ such that $\mathcal{Q} = \bigcup_j \mathcal{Q}_j$ with $\mathcal{Q}_j \cap \mathcal{Q}_k = \emptyset$ for $j \neq k$. Several approaches for quadrature grid batching have been suggested in various contexts.[10,103,106] In this work, we utilize an octree-based partition scheme used in previous studies by the authors.[17] In general, the octree approach will yield more spatially local quadrature batches than other approaches,[10] but it will also yield batches of irregular sizes, potentially leading to load imbalance between tasks. We discuss the implications of this irregularity in the following. Given a partitioning $\{\mathcal{Q}_j\}_{j=1}^{N_{\text{batch}}}$, one may construct sets of important quantities that are approximately considered non-negligible for a particular integrand (e.g., $\mathbf{K}$ or $\mathbf{V}^{xc}$). The simplest of these sets, which is required for both numerical integrands considered in this work, pertains to negligible basis functions,

$$\mathcal{B}_j = \{\phi_\mu \mid \exists \mathbf{r}_i \in \mathcal{Q}_j \quad \text{s.t.} \quad |\phi_\mu(\mathbf{r}_i)| \geq \varepsilon_B\}, \tag{41}$$

where ε_B is a basis tolerance. For GTO bases, Eq. (41) may be quickly determined by encircling each basis function with a sphere of radius r_μ^{cut} such that $|\phi_\mu(\mathbf{r}-\mathbf{R}_\mu)|\langle\varepsilon_B \ \forall|\mathbf{r}-\mathbf{R}_\mu|\rangle r_\mu^{\text{cut}}$ and only keeping those elements of $\mathcal{B}$ for which their non-negligible sphere spatially overlaps $\mathcal{Q}_j$.[10,17,89,106] This process formally scales $\mathcal{O}(N_b N_{\text{batch}})$ as each basis shell must be checked against each $\mathcal{Q}_j$. It is worth noting that in scenarios where the nuclei remain stationary for several Fock builds, e.g., in a self-consistent field (SCF) optimization, $\mathcal{B}_j$ need only be computed once and reused for subsequent iterations. This process of basis screening is the primary mechanism by which sparsity may be exploited in the numerical integration of $\mathbf{V}^{xc}$.[106] For example, given $\mathcal{B}_j$, the compute-intensive Eq. (35) becomes

$$F_{\mu i}^{xc,(j)} = \sum_{\nu\in\mathcal{B}_j} D_{\mu\nu}^{xc,(j)}\Phi_{\mu i}^{(j)}, \qquad \mu \in \mathcal{B}_j, \qquad i \in \mathcal{Q}_j, \tag{42}$$

where $\mathbf{D}^{xc,(j)}$, $\mathbf{F}^{xc,(j)}$, and $\mathbf{\Phi}^{(j)}$ are contiguous batch-local matrices pertaining to $\mathcal{Q}_j$.

The case is significantly different for the sn-K method as basis function and integral screening alone can only produce $\mathcal{O}(N^2)$ scaling.[119] Several methods have been suggested for density-dependent screening in the sn-K method.[10,87–89,120] Due to its simple form and demonstrated ability to generate sufficient sparsity for near-linear scaling, we adopt the sn-LinK screening approach reported in the work of Laqua *et al.*[10] in this study, though we note that other schemes could be employed with only minor modifications of the overall algorithm design. In the sn-LinK method, a list of basis pairs, $\mathcal{V}_j$, is selected for each $\mathcal{Q}_j$ such that

$$\mathcal{V}_j = \{(\phi_\mu, \phi_\nu) \quad | \quad \varepsilon_{\mu\nu}^{E(j)} \geq \varepsilon_E \vee \varepsilon_{\mu\nu}^{K(j)} \geq \varepsilon_K\}, \tag{43}$$

where ε_K and ε_E are tolerances that screen shell pair contributions to $\mathbf{K}$ and $\mathrm{Tr}[\mathbf{KD}]$ (the exact exchange energy), respectively, and

$$\varepsilon_{\mu\nu}^{E(j)} = \tilde{F}_\mu^{K,(j)}\tilde{F}_\nu^{K,(j)}\tilde{A}_{\mu\nu}, \tag{44}$$

$$\varepsilon_{\mu\nu}^{K(j)} = \max\left(\tilde{F}_\mu^{K,(j)}, \tilde{F}_\nu^{K,(j)}\right)\tilde{\phi}^{(j)}\tilde{A}_{\mu\nu}, \tag{45}$$

$$\tilde{F}_\mu^{K,(j)} = \sum_{\nu\in\mathcal{B}_j} |D_{\mu\nu}|\tilde{\Phi}_\nu^{(j)}, \tag{46}$$

$$\begin{aligned} \tilde{A}_{\mu\nu} &= \max_{i\in\mathcal{Q}_j} |A_{\mu\nu i}|, \\ \Phi_\nu^{(j)} &= \max_{i\in\mathcal{Q}_j} |\sqrt{w_i}\Phi_{\nu i}|, \\ \tilde{\phi}^{(j)} &= \max_{i\in\mathcal{Q}_j} \sqrt{w_i}\sum_{\mu\in\mathcal{B}_j} |\Phi_{\mu i}|. \end{aligned} \tag{47}$$

To avoid recomputation of the 3c-CP integrals in the screening procedure, $\tilde{A}_{\mu\nu}$ is approximated with global upper-bound estimates from the appendix of Ref. 121 in practice.[10] We refer the reader to the work of Laqua *et al.*[10] for further details pertaining to the veracity of this screening procedure for sn-K. Given $\mathcal{V}_j$, block-sparse expressions similar to Eq. (42) can be derived for sn-K, e.g.,

$$F^{K,(j)}_{\lambda i} = \sum_{\nu \in \mathcal{B}_j} D^{K,(j)}_{\lambda\nu} \Phi^{(j)}_{\nu i},$$
$$G^{K,(j)}_{\mu i} = \sum_{(\cdot,\lambda)\in\mathcal{V}_j} w_i A_{\mu\lambda i} F^{K,(j)}_{\lambda i}, \tag{48}$$
$$\mu,\lambda \in \mathcal{V}_j, \qquad i \in \mathcal{Q}_j.$$

The $(\cdot,\lambda)$-sum pertaining to $G^{K,(j)}_{\mu i}$ indicates only inclusion of terms for which $\exists \phi_\mu$ such that $(\phi_\mu, \phi_\lambda) \in \mathcal{V}_j$. Both the $\mathcal{B}$-collision detection and the evaluation of performance critical $\mathcal{V}_j$-intermediates [e.g., Eq. (46) can be implemented as a single, large dimension-GEMM over tasks] are executed on the GPU in this work.

There are several key differences in screening processes for sn-K in comparison with the XC integration:

1. As this procedure is density-dependent, $\mathcal{V}_j$ must be reevaluated for each new density and its cost cannot generally be amortized over, e.g., an SCF optimization.
2. While the matrix multiplication in Eq. (46) can be restricted over $\nu \in \mathcal{B}_j$, the μ-index *must* be over the entire basis set $\mathcal{B}$ in order to check for all possible nontrivial elements of $\mathcal{V}_j$. Therefore, Eq. (46) formally scales $\mathcal{O}(N_b N_{\text{batch}})$ assuming $\mathcal{B}_j$ can be screened to an amortized constant.[106] While this formally scales the same as $\mathcal{B}$-screening, it will be accompanied by a much larger prefactor (GEMM vs scalar collision detection).
3. Equation (43) scales $\mathcal{O}(N_b N_{\text{batch}})$ as each non-negligible shell pair [which asymptotically scales $\mathcal{O}(N_b)$] must be checked against every $\mathcal{Q}_j$.

We discuss the implications of this screening procedure on the overall performance and scalability of numerical integration procedures in Sec. III.

F. Load balancing and parallel numerical integration

In addition to enabling the screening of basis functions and operator integrals, the locality of the numerical integration procedure considered in this work is also particularly advantageous for distributed memory implementations.[17,24,31] As the batch-local quantities discussed in Subsection II E are independent, they may be executed concurrently and assembled *a posteriori* via collective reduction operations to form final integrands. The general distributed memory scheme explored in this work is a simple three-step process (Algorithm 1) that is the same for both the sn-K and XC integrations.[17]

Due to the varying extent to which sparsity can be realized between differing spatial regions, there is a significant potential for load imbalance in parallel implementations of molecular integration that rely on grid partitioning.[17,24,25] The optimal task assignment problem for irregular work is NP-Hard,[122] but reasonable solutions can be obtained using heuristic-driven methods. A recent distributed memory DFT application[24] has adopted a dynamic load balancing scheme to address this problem. At large processor counts, dynamic load balancing algorithms rely on the ability to overlap the costs associated with the generation, assignment, and communication of tasks with their execution on local processors. Given the fine-granularity and large number of tasks generated by the octree-batching algorithm discussed in Sec. II E, the communication costs associated with the memory requirement of each task (e.g., $\mathcal{Q}_j$, $\mathcal{B}_j$ and $\mathcal{V}_j$) far outweigh their associated computational work. Instead, we have developed a *static* load balancing procedure for molecular integration that allows for *a priori* distribution of work without the need to carefully balance the overlap of local work and task communication. In this static load balancing scheme, a heuristic cost, W_j, is associated with each task and is subsequently assigned to a rank via a greedy algorithm illustrated in Algorithm 2. For the XC integration, the work heuristic is given by[17]

$$W_j = \left(|\mathcal{B}_j| \times (9 + 2|\mathcal{B}_j|) + N_A^2 + 3\right) \times |\mathcal{Q}_j|. \tag{49}$$

Algorithm 2 is a replicated procedure that allows for near optimal task assignment given work heuristics that properly rank the relative computational costs between different tasks. As such, while being communication-free, it constitutes a scaling bottleneck due to Amdahl's law. Similar algorithms based on prefix-sums have been suggested for the distributed generation of tasks[122] However, we have found that because Algorithm 2 is able to globally optimize task assignment (via the sort of all tasks on W_j), it generally is able to obtain a more optimal task partitioning than those based on prefix-sums. Due to the fact that $\mathcal{B}_j$ in particular can be computed once-per-nuclear configuration, the scaling bottleneck associated with the replicated nature of Algorithm 2 can be amortized over several Fock builds for the XC integration.

For the sn-K integration, the problem becomes slightly more difficult. While it would be possible to generalize Eq. (49) to account for $|\mathcal{V}_j|$, replication of Eqs. (43) and (46) on each processor for all tasks would be a considerable bottleneck due to the steep scaling

ALGORITHM 1. General scheme for the parallel, GPU accelerated evaluation of $\mathbf{V}^{xc}$ or $\mathbf{K}$.

1: **Input:** Density matrix $\mathbf{D}$, basis set $\mathcal{B}$.
2: **Output:** Desired integrand $\mathbf{X} \in \{\mathbf{V}^{xc}, \mathbf{K}\}$
3: $\mathcal{Q}_{local} \leftarrow$ generate balanced local quadrature batches ▷ Algorithm 2
4: Preallocate fraction of device memory.
5: Send quadrature independent data (e.g., $\mathbf{D}$) to the device ▷ cpu/gpu
6: $\mathbf{X}_{local} \leftarrow \mathbf{0}$ ▷ gpu
7: **while** $\mathcal{Q}_{local} \neq \emptyset$ **do**
8: $\mathcal{Q}_{batch} \leftarrow$ subset of $\mathcal{Q}_{local}$ s.t. $\mathbf{X}$-intermediates saturate device memory. ▷ cpu
9: Send $\mathcal{Q}_{batch}$ data (e.g., $\{\mathcal{B}_j\}_{\text{local}}$, $\{\mathcal{V}_j\}_{\text{local}}$) to device ▷ cpu/gpu
10: $\mathcal{Q}_{local} \leftarrow \mathcal{Q}_{local} \backslash \mathcal{Q}_{batch}$ ▷ cpu
11: $\mathbf{X}_{local} \leftarrow \mathbf{X}_{local}$ + $\mathcal{Q}_{local}$ contributions to $\mathbf{X}$ ▷ gpu
12: **end while**
13: $\mathbf{X} \leftarrow$ `(All)reduce`$(\mathbf{X}_{local})$ ▷ collective

ALGORITHM 2. Replicated load balancing algorithm for distributed memory molecular integration.

1: **Input:** Quadrature batches $\{\mathcal{Q}_j\}_{j=1}^{N_{\text{batch}}}$
2: **Output:** Local quadrature batches $\mathcal{Q}_{\text{local}} \subset \{\mathcal{Q}_j\}$
3: `world_size` ← Number of ranks
4: `world_rank` ← Index of this rank
5: $\mathcal{T} \leftarrow [\,]$
6: **for** $j \in [1, N_{\text{batch}}]$ **do**
7: $\quad \mathcal{T}_j = (\mathcal{Q}_j, W_j)$ ▷ Eq. (49)
8: **end for**
9: $\mathcal{T} \leftarrow$ sort $\mathcal{T}$ by W_j
10: $\mathcal{G} \leftarrow$ `zeros(world_size)`
11: $\mathcal{Q}_{\text{local}} \leftarrow [\,]$
12: **for** $j \in [1, N_{\text{batch}}]$ **do**
13: $\quad I \leftarrow \arg\min_J \mathcal{G}_J$ ▷ global
14: $\quad \mathcal{G}_I \leftarrow \mathcal{G}_I + W_j$ ▷ global
15: $\quad$ **if** $I =$ `world_rank` **then**
16: $\quad\quad \mathcal{Q}_{\text{local}} \leftarrow \mathcal{Q}_{\text{local}} \cup \mathcal{Q}_j$ ▷ local
17: $\quad$ **end if**
18: **end for**
19: **return** $\mathcal{Q}_{\text{local}}$

and prefactors associated with sn-K screening discussed in Subsection II E. Given an initial work partitioning generated by, e.g., Eq. (49), $\mathcal{V}_j$ may be computed locally and could in principle be rebalanced according to the distributed prefix-sum algorithms previously mentioned. Apart from generally obtaining less-optimal work partitions than Algorithm 2, for hybrid DFT simulations requiring the evaluation of both $\mathbf{V}^{xc}$ and $\mathbf{K}$, this procedure would require rebalancing the work distribution at every SCF step, which would in turn incur significant communication overhead. Instead, we have chosen to reuse the same task distribution for both sn-K and the XC integration based on Algorithm 2 and Eq. (49). We examine the veracity of this reuse to yield balanced work for both sn-K and the XC integration in Sec. III.

If screened sufficiently well, the dimensions of the packed batch-local sub-matrices in Eqs. (31) and (32) will be small, which means that the GEMM operations in, e.g., Eqs. (42) and (48) will be of low dimension. While this screening leads to a significant decrease in the required computational work, any particular GEMM operation (or other kernel invocation) will not constitute enough work to occupy the resources of a GPU. While it is possible to concurrently execute GPU kernels via streaming (as has been explored in other works for GPU accelerated sn-K integration[10]), the large number of kernels invoked for large systems partitioned by the octree-method would incur significant kernel launch overhead. As has been demonstrated in previous studies regarding the XC integration,[17,18] the use of *batched* kernels to concurrently execute logically identical tasks can lead to large performance improvements over concurrent stream injection for fine task granularity. The challenge for DFT applications is in that the dimensions of $\mathcal{B}_j$ and $\mathcal{V}_j$ can vary drastically in different spatial regions. For the evaluation of batched GEMM operations [e.g., Eqs. (42) and (48) batched over j indices], standard solutions to the problem exist in community GPU linear algebra libraries in the form of variable-sized batched BLAS.[123,124]

For sn-K, the batched evaluation of Eq. (39) is a straightforward application of variable-sized batched GEMM as was applied to the formation of Eq. (42) for the XC integration. Batching of Eq. (38) can be achieved hierarchically over three dimensions: (1) $\{\mathcal{Q}_j\}$, (2) $(\mu, \nu) \in \mathcal{V}_j$ within $\mathcal{Q}_j$, and at the lowest level (3) $i \in \mathcal{Q}_j$. The kernel-level details for (3) are given in the Appendix. For any batched kernel algorithm, the natural challenge is to decide which task to batch together to execute concurrently on the device. For the algorithms presented here, we make the simple choice of executing as many tasks as will fit into device memory (Algorithm 1). While this choice does mitigate kernel launch overhead and allows for the overlap of host data manipulations with GPU activity, it also introduces significant pressure on the GPU scheduler to effectively execute large numbers of independent tasks with variable amounts of work. However, we have seen that, for DFT applications, this device-saturation approach is capable of yielding excellent utilization of GPU resources on an array of modern GPU hardware.[17,18]

In the large processor limit, the reduction step poses a considerable bottleneck for large N_b as the message size for the `(All)reduce` operation scales $\mathcal{O}(N_b^2)$. While standard MPI collectives are often sufficient for small N_b, we have found that for NVIDIA hardware, the NVIDIA Collective Communication Library (`NCCL`)[125] Allreduce primitive can yield significant strong scaling improvements for large N_b. `NCCL` provides collective primitives with an API similar to that of MPI with the addition of a CUDA stream argument that allows for its injection into asynchronous workflows. The Allreduce algorithm and exact choice of communication routes are selected by `NCCL` at runtime based on the system topology allowing for optimal performance on a wide range of hardware configurations and message sizes. We will examine the comparison of MPI and `NCCL` collectives for this application in Sec. III.

III. NUMERICAL RESULTS

The methods presented in this work are made publicly available as a part of the open-source `GauXC` library for exascale Gaussian basis DFT (XC and sn-K)[17,18,104] and the `LibintX` library for Gaussian AO integrals (DF-J-engine).[96] Each of these libraries was developed to be a modular, reusable GPU-based component under the NWChemEx Exascale Computing Project and is freely available on GitHub[99,126] for use in other electronic structure packages. At present, these methods have been integrated into the `NWChemEx`[127] and `MPQC`[128] computational chemistry software packages. In the present study, all numerical results were obtained using `MPQC`; however, as the focus of this work is on the parallel construction of hybrid DFT Fock matrices, the performance characterizations presented here are expected to be representative of any software integration involving these libraries. All numerical experiments were carried out on the GPU partition of the Perlmutter (PM) supercomputer at the National Energy Research Scientific Computing Center (NERSC). Each PM GPU node has 4 NVIDIA A100 GPUs (40 GB HBM2e RAM) and 1 AMD EPYC 7763 CPU (64 cores @ 3.5 GHz). The GPUs within a PM node are connected via NVLink and are connected to the CPU via the peripheral component interconnect express (PCI-e) interface. Internode communication is facilitated through the HPE Slingshot 11 interconnect. All numerical experiments were configured with 1 MPI rank per GPU (4 ranks per node, 16 threads per rank) and the threads of

TABLE I. Representative systems considered in this study. N_A is the number of atoms, N_b is given for Cartesian 6-31G(d) and N_{aux} for def2-tzvp-j.

Molecule	N_A	N_b	N_{aux}
Taxol	113	1 032	3 599
Olestra	453	3 181	11 633
Crambin	642	5 559	19 500
Ubiquitin	1231	10 292	36 419

each MPI rank were bound to respective non-uniform memory access (NUMA) domains (of which there are 4 per PM node). GPU Batched BLAS operations for the sn-K and XC integrations were provided by the MAGMA library.[123,124] All domain-specific GPU kernels for the DF-J and sn-K algorithms were implemented in CUDA, with the latter having been integrated into the performance portable numerical integration software infrastructure developed in Ref. 18.

As have been utilized in other studies,[17,24] we have included experiments with four test molecules, the specifics of which can be found in Table I. The geometries for Taxol, Olestra, and Ubiquitin were taken from Ref. 17 and the Crambin geometry was taken from Ref. 24. Each of these geometries can be found in the supplementary material. All experiments utilize the Pople 6-31G(d) basis set[129–131] with Cartesian d-functions for $\mathcal{B}$, def2-tzvp-j fitting basis for DF-J,[132], and the PBE0 hybrid XC functional.[133] GPU evaluation of the XC functional was performed using the open-source `ExchCXX` library.[134] All calculations were performed with $\varepsilon_B = \varepsilon_E = \varepsilon_K = 10^{-9}$ and with a maximum grid partition size of 4096. Evaluation of 3-center integrals in Eqs. (10) and (12) is screened using user-provided screener that is provided by the user via the `LibintX` API. All computational experiments used the default (Schwarz) screener of 3-center AO integrals omitting evaluation of integral sets below the 64-bit floating point epsilon ($\approx 2 * 10^{-16}$). All performance data have been generated using SCF density matrices converged to an orbital gradient norm of $\varepsilon_{SCF} = 10^{-10}$. The results of these SCF optimizations can be found in the supplementary material.

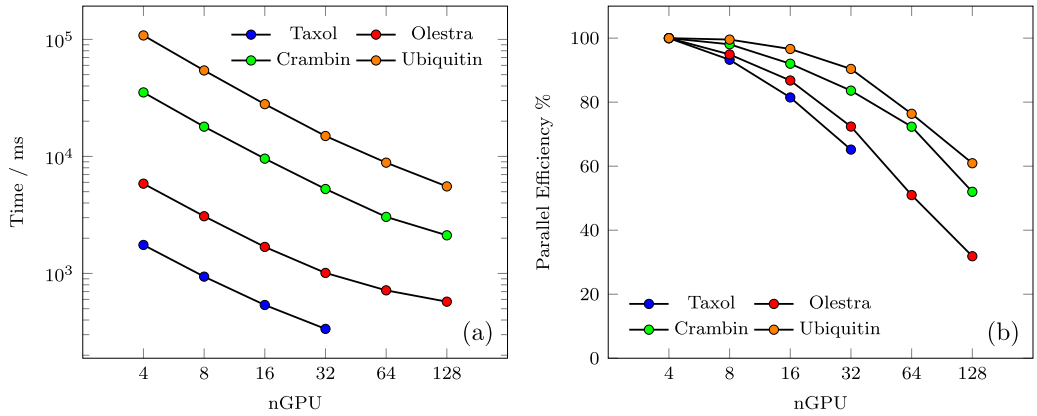

FIG. 1. Strong scaling of distributed memory Fock algorithm for `Grid1`. (a) Overall timings and (b) parallel efficiency relative to single-node performance.

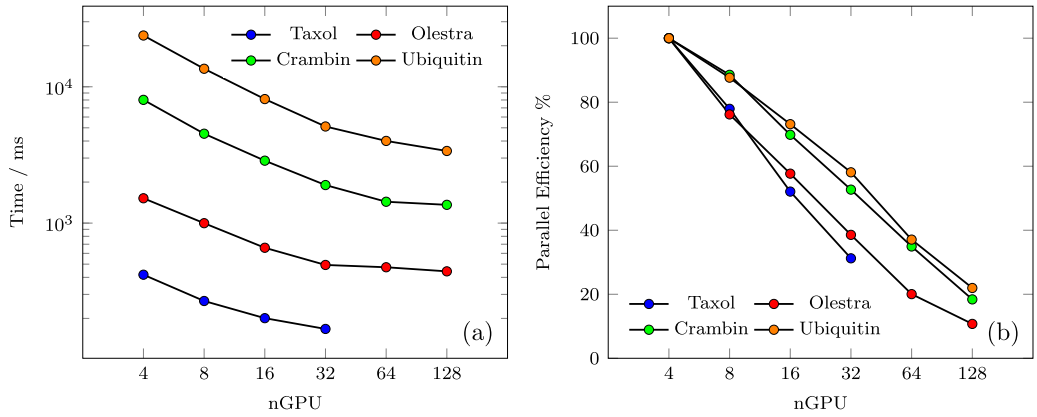

FIG. 2. Strong scaling of distributed memory Fock algorithm for `Grid2`. (a) Overall timings and (b) illustration of the parallel efficiency relative to single-node performance.

TABLE II. Spherical atomic quadratures consisting of N_{rad} radial points and N_{ang} angular points used in this work. Angular resolution has been pruned according to the scheme presented in Ref. 110.

Grid name	N_{rad}	N_{ang}
`Grid1`	50	302
`Grid2`	30	110

Figures 1 and 2 show the strong scaling performance and parallel efficiency (PE) of the overall Fock matrix construction for two different integration grids (summarized in Table II). The same grid is used for both XC and sn-K. The accuracy of these grids relative to direct ERI assembly of the exact exchange matrix for the two smaller problems can be found in the supplementary material and we refer the reader to Refs. 10 and 14 for more comprehensive discussions pertaining to the overall accuracy and single-node performance of the sn-K and DF-J methods. Here, PE is measured relative to single-node (4 GPU) performance and all results are presented with the `NCCL` reduction optimization discussed in Sec. II F. For the larger grid (`Grid1`), we see that excellent PE (>80%) is achieved for all considered problems out to 16 GPUs and maintains a very reasonable PE (>70%) out to 32 GPUs except for Taxol, which exhibits 64%. It is for this reason that we have not pursued performance analysis for Taxol past 32 GPUs. This result is comparable with the achieved PE in other recent distribution memory Fock algorithms at similar GPU counts.[23,24] For Crambin, >70% PE is maintained out to 64 GPUs and achieves 52% PE at 128 GPUs. For the largest problem (Ubiquitin), >90% PE is maintained out to 32 GPUs, >75% PE out to 64 GPUs, and achieves 61% PE at 128 GPUs. For small problems (Taxol and Olestra) and the smaller grid (`Grid2`), strong scaling stagnation is encountered immediately. Apart from the self-evident increase in available local work exhibited by the larger test problems and grids, the smaller test problems expose bottlenecks (such as communication and load imbalance) in the relative performance and scalability of the individual Fock matrix components. The minimum time to solution for these problems, including individual component timings, are given in Table III.

Figures 3 and 4 illustrate the strong scaling of the **J**, **K** (sn-K), and $\mathbf{V}^{xc}$ components relative to the total Fock formation for the Olestra and Ubiquitin test cases at various grid sizes. The textual annotations denote the wall time percentage of each component relative to the overall Fock build. For the Ubiquitin test case (Fig. 3), sn-K dominates the overall Fock build by a large margin at all GPU counts, and the strong scaling of the Fock build is virtually identical to that of sn-K (as illustrated by the parallelity of the scaling plots). As such, the overall scaling behavior of the Fock build is insensitive

TABLE III. Minimum time to solution for considered systems on 128 NVIDIA A100 GPUs. All times are given in milliseconds (ms).

		`Grid1`			`Grid2`		
Molecule	DF-J	XC	sn-K	Fock	XC	sn-K	Fock
Taxol	63.2	95.1	256.9	423.6	69.4	107.5	240.0
Olestra	113.5	363.6	565.9	1043.3	320.5	438.0	872.0
Crambin	177.8	1043.4	2226.4	3448.8	968.6	1512.6	2659.0
Ubiquitin	378.8	2443.1	5653.1	8465.6	2268.0	3674.0	6320.8

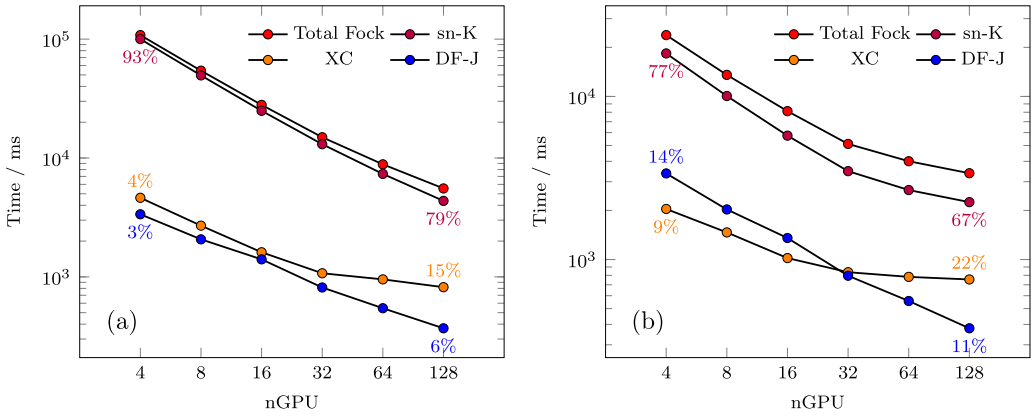

FIG. 3. Strong scaling of individual Fock matrix components for Ubiquitin 6-31G(d)/def2-tzvp-j/PBE0 using (a) `Grid1` and (b) `Grid2` for the XC and sn-K integration. Annotations depict the relative contribution of each component to the overall Fock timings.

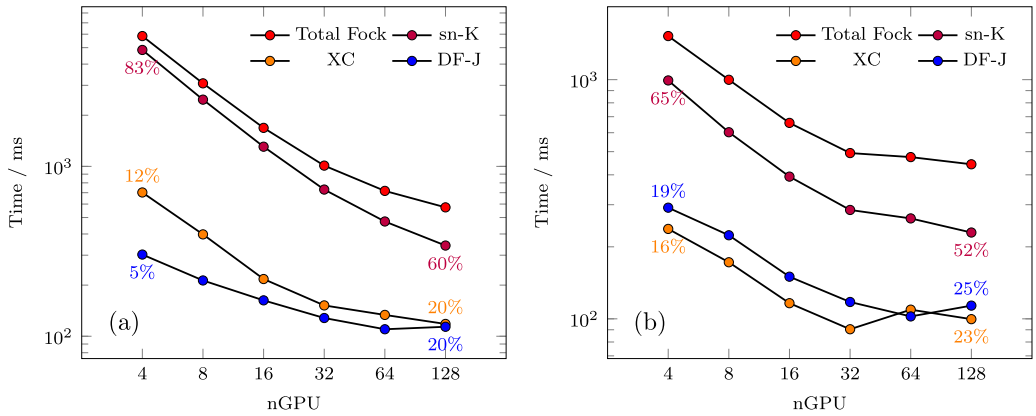

FIG. 4. Strong scaling of individual Fock matrix components for Olestra 6-31G(d)/def2-tzvp-j/PBE0 using (a) `Grid1` and (b) `Grid2` for the XC and sn-K integration. Annotations depict the relative contribution of each component to the overall Fock timings.

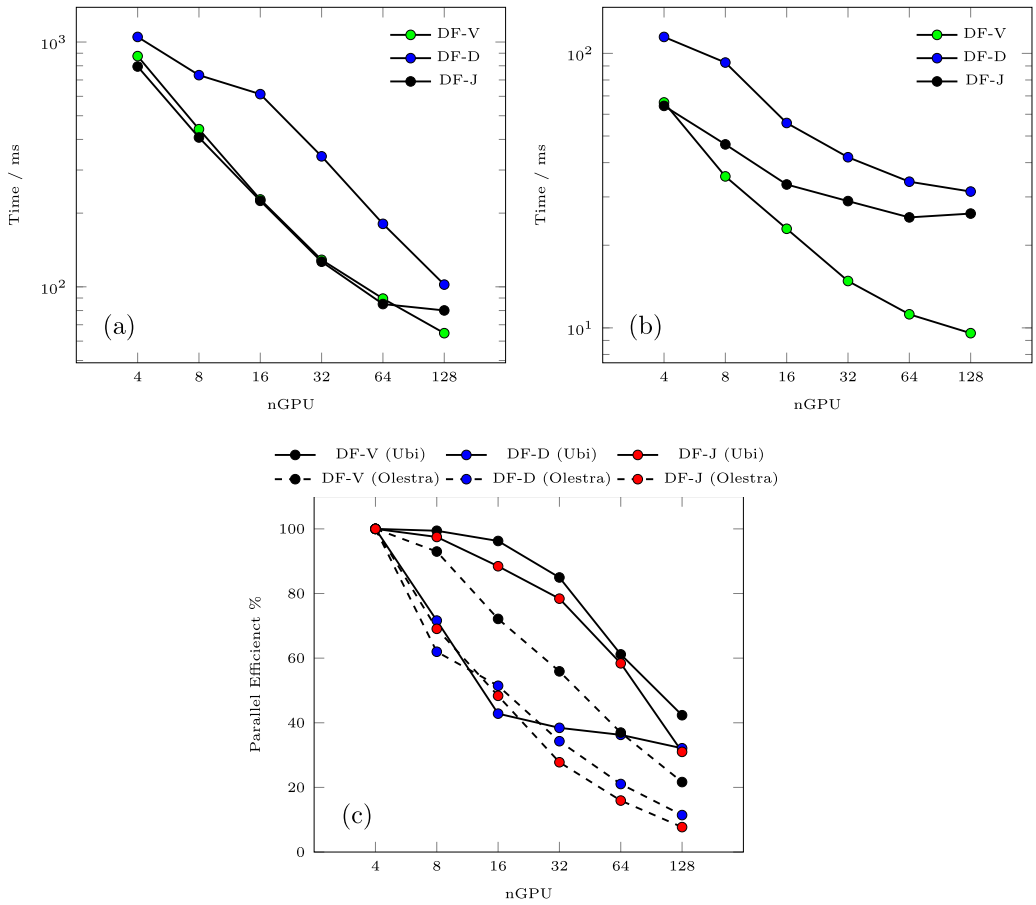

FIG. 5. Component strong scaling of the distributed DF-J algorithm presented in Sec. II B for (a) Ubiquitin and (b) Olestra 6-31G(d)/def2-tzvp-j. (c) Illustration of the parallel efficiencies of these components for both problems.

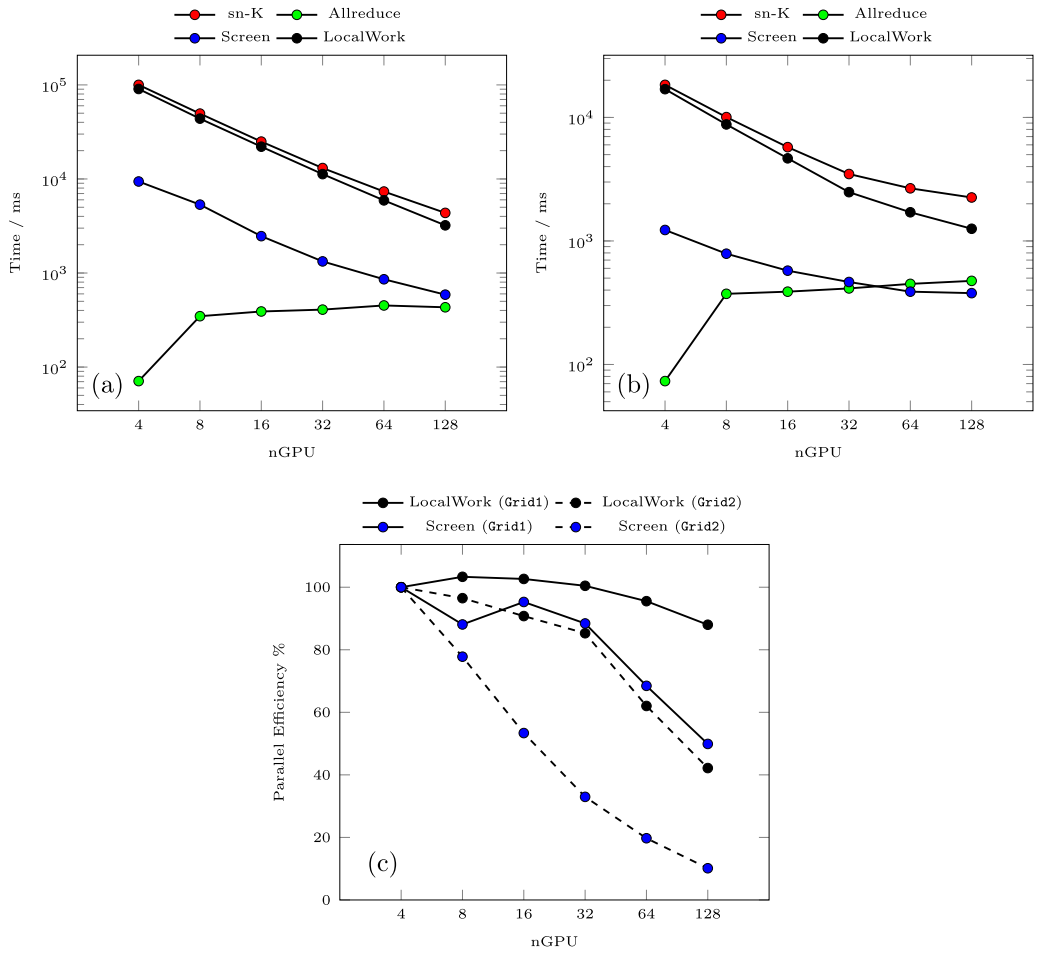

FIG. 6. Component strong scaling of the distributed sn-K algorithm presented in Algorithm 2 for Ubiquitin 6-31G(d) using (a) `Grid1` and (b) `Grid2` numerical quadratures. (c) Illustration of the parallel efficiency of these components for both problems.

to the scaling behaviors of **J** and XC. The opposite case is illustrated for Olestra (Fig. 4), where the **J**-build and XC integration constitute a much more considerable portion of the overall Fock build than was exhibited for Ubiquitin, particularly at large GPU counts. As such, we can conclude that the scaling behaviors of **J** and XC have a much more measurable impact on the overall strong scaling of the Fock build for smaller systems.

The performance and scaling behavior of the XC integration via the algorithms presented in this work (modulo the `NCCL` reduction) have been explored in other works.[17,18] In Fig. 5, we examine the scalability of the three primary components of the DF-J algorithm: `DF-V` [Eq. (10)], `DF-D` [Eq. (11)], and `DF-J` [Eq. (12)]. For Ubiquitin, the integral-driven `DF-V` and `DF-J` exhibit >65% PE for up to 32 GPUs whereas the scaling of the TiledArray GEMV in `DF-D` stagnates immediately. Lack of scaling in the latter can be attributed to the lack of computational intensity exhibited by distributed GEMV. For Olestra, `DF-D` and `DF-J` exhibit immediate scaling stagnation while `DF-V` demonstrates >50% PE for up to 32 GPUs.

For the sn-K algorithm (Fig. 6), we examine the strong scaling of the local integration (`LocalWork`), shell pair screening (`Screen`), and collective reduction (`Allreduce`) for both grid sizes. Host-to-device and device-to-host data movement timings are included in the `LocalWork` timings. For `Grid1`, the overall sn-K integration is dominated by the local integration at all considered GPU counts. However, for `Grid2`, the slight deviation of the sn-K scaling from the `LocalWork` scaling can be attributed to the growing importance of the reduction and screening operations in the strong scaling limit. In Fig. 6(c), we see that for `Grid1`, the `LocalWork` timings exhibit near perfect strong scaling >95% out to 64 GPUs and degrades only to 88% at 128 GPUs. The `LocalWork` timings for `Grid2` and the `Screen` timings `Grid1` exhibit reasonable scalability (>60%) out to 32 GPUs but quickly stagnate thereafter. These results indicate that the reuse of the XC scheduling heuristic discussed in Sec. II F is viable for the *a priori* scheduling of sn-K integration tasks given enough distributable work.

While remaining communication-free, it is important to note that the chosen work distribution scheme for sn-K requires the

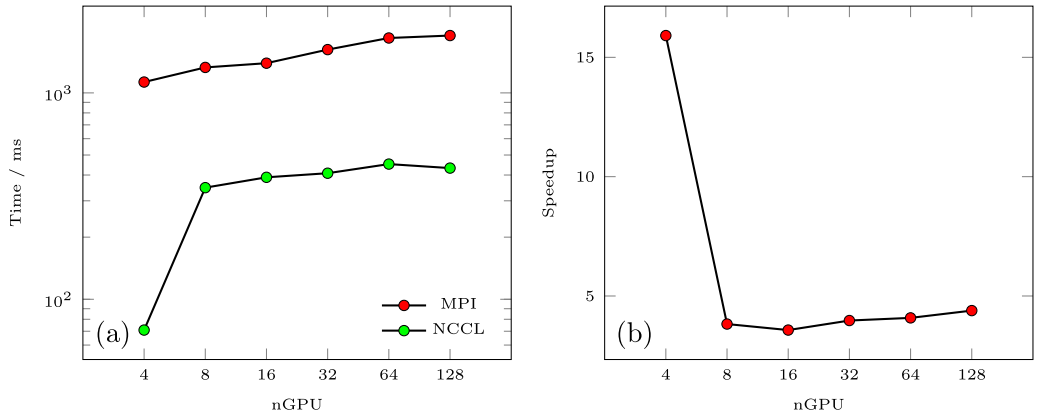

FIG. 7. Comparison of Cray MPI and `NCCL` `Allreduce` performance for Ubiqutin 6-31G(d). (a) Strong scaling of `Allreduce` for these two libraries and (b) illustration of the speedup of `NCCL` over Cray MPI.

reevaluation of Eqs. (43) and (46) for $\mathcal{Q}_{\text{local}}$ at each SCF step. Due to the fact that the scaling of each of these terms is linear in N_g, it is expected that the work associated with sn-K screening will decrease as the quadrature batches are distributed among independent ranks. However, in the large processor limit where there are very few tasks per rank, there remains a large prefactor associated with the formation of $\mathcal{V}_j$, which scales as $\mathcal{O}(N_b)$. As such the scaling stagnation of the `Screen` timings can be attributed to the $\mathcal{O}(N_b)$ prefactor of the screening procedure overtaking the generation of quadrature batches in the large processor limit.

It is expected that the cost of the reduction operation will grow with growing processor counts as the connectivity of both `Allreduce` communication graphs grows logarithmically with the number of processors. To demonstrate the performance of the `NCCL` reduction optimization, we compare the Ubiquitin XC/sn-K reduction with `MPI_Allreduce` from the Cray MPI library in Fig. 7. As we can see, `NCCL` improves the overall communication cost by about a factor of 4 at all GPU counts, except for 1 node at which the speedup is 15. The reason for this speedup can be attributed to the fact that the 4 GPUs on a PM are connected via NVLink, which allows for a very fast intranode reduction to take place prior to triggering internode communication over the Slingshot interconnect. This fast intranode reduction is clearly present in the 1 node case where no internode communication is triggered. When using the MPI reduction primitives, sn-K and XC exhibit a 20% reduction in parallel efficiency due to the growing relative communication cost in the strong scaling limit.

The consistent stagnation of strong scaling behavior as problem size (e.g., N_g and/or N_b) decreases can be attributed to two primary factors: (1) communication overhead and (2) a lack of divisible work to be distributed among independent ranks leading to load imbalance in the large processor limit. The communication overhead (Fig. 7) represents a slow-growing, nearly constant cost with processor count, which means that even in the case of perfect strong scaling for the local work of the Fock build, communication will eventually dominate the strong scaling behavior. While these scaling inefficiencies are considerable in the large processor limit, it is important to contextualize the scaling behavior of these smaller problems in reference to their overall wall time at this scale (Table III). For example, while the DF-J algorithm becomes a considerable wall time contribution for the smaller problems in the strong scaling limit, the overall wall time for this operation is <0.4 s for all problems considered. Furthermore, the 128 GPU timing for sn-K is under <=0.6 s for Olesta and Taxol, <3 s for Crambin, and <6 s for Ubiquitin. Due to the small timing margins at this scale, it is unlikely that further algorithmic optimization would overcome inherent bottlenecks to significantly change the presented scaling results.

IV. CONCLUSIONS

In this work, we have presented a set of GPU accelerated, distributed memory algorithms for the evaluation of the performance critical Coulomb and exact exchange matrices for Gaussian basis DFT. For the Coulomb matrix, we developed a DF-based J-engine implementation capable of efficient execution on distributed memory heterogeneous platforms. This work extends our recent developments of algorithms for evaluation of 3-center integrals on accelerated architectures.[96] For the exact exchange matrix, we have developed a distributed memory sn-K algorithm that extends recently developed numerical integration methods for the Gaussian basis XC potential.[17] These methods were implemented in the open-source `LibintX` and `GauXC` libraries and are publicly available on GitHub.[99,126] We have demonstrated the absolute and strong scaling performance of the presented algorithms for a representative set of molecular systems out to 128 NVIDIA A100 GPUs on the Perlmutter supercomputer. For the largest problem considered (Ubiquitin), total Fock construction via our methods exhibited >90% parallel efficiency out to 32 GPUs and achieved 61% efficiency out to 128 GPUs for a total Fock matrix duration of ~8.5 s. While degraded strong scalability was exhibited for the smaller problems considered due to communication overhead and load imbalance, the magnitude of their effects on overall scalability is magnified relative to low execution time of these operations and is small on an absolute scale.

While results of the present work indicate a promising future for distributed memory, GPU accelerated algorithms for Gaussian

basis DFT, there remain several areas for exploration in future work. First, although SCF optimizations were performed to obtain converged density matrices, no performance optimizations pertaining to the overall SCF procedure were pursued in this work. The performance of the overall SCF procedure depends on a number of confounding factors, including the choice of optimization method (e.g., diagonalization-based, direct-minimization) as well as the quality of the optimizer and initial guess. Although the problems considered in this work are of considerable size for current state-of-the-art Gaussian basis DFT methods, the overall problem dimensions are of insufficient scale to leverage the same degree of concurrency in the SCF as the presented Fock matrix algorithms. As such, future work is required to properly optimize the performance of SCF procedures on massively parallel architectures. Efficient evaluation of the Coulomb potential described here can be combined straightforwardly with fast $[\mathcal{O}(N)]$ approaches for evaluation of the Coulomb potential, such as the fast multipole method (FMM) that can be applied in both nonperiodic[80] and periodic[135,136] settings as well as the Ewald summation[137] for periodic systems. While the seminumerical method was only applied to the problem of exact exchange in this work, similar approaches have been developed for the treatment of correlated many-body methods as well.[138,139] Pursuance of many-body extensions to the presently developed distributed memory sn-K method will be the subject of future work by the authors. Finally, as has recently been demonstrated for the XC integration,[18] the modular nature `GauXC` library allows for rapid development of performance portable DFT methods by separating the implementation details of performance critical kernels from their inclusion in high-level workflows, thereby exposing the highest potential and flexibility for targeting current and emerging accelerator hardware with minimal developer effort. The pursuance of performance portable DF-J and sn-K algorithms targeting emerging AMD and Intel GPUs will also be pursued by the authors in upcoming work.

SUPPLEMENTARY MATERIAL

See the supplementary material for molecular geometries and SCF optimization results for the systems considered in this study.

ACKNOWLEDGMENTS

The authors thank Norm M. Tubman for proofreading the draft of this manuscript and providing useful feedback. This research was supported by the Exascale Computing Project (Grant No. 17-SC-20-SC), a collaborative effort of the U.S. Department of Energy Office of Science and the National Nuclear Security Administration. This research used resources of the National Energy Research Scientific Computing Center (NERSC), a U.S. Department of Energy Office of Science User Facility located at Lawrence Berkeley National Laboratory, operated under Contract No. DE-AC02-05CH11231 using NERSC Award No. ASCR-ERCAP-M3946.

AUTHOR DECLARATIONS

Conflict of Interest

The authors have no conflicts to disclose.

Author Contributions

David B. Williams-Young: Investigation (equal); Methodology (equal); Software (equal); Writing – original draft (equal); Writing – review & editing (equal). **Andrey Asadchev**: Investigation (equal); Methodology (equal); Project administration (equal); Software (equal); Writing – original draft (equal); Writing – review & editing (equal). **Doru Thom Popovici**: Investigation (equal); Methodology (equal); Software (equal); Writing – original draft (equal); Writing – review & editing (equal). **David Clark**: Software (equal); Writing – original draft (equal); Writing – review & editing (equal). **Jonathan Waldrop**: Investigation (supporting); Software (supporting); Writing – original draft (equal); Writing – review & editing (equal). **Theresa L. Windus**: Funding acquisition (lead); Project administration (equal); Writing – original draft (equal); Writing – review & editing (equal). **Edward F. Valeev**: Investigation (equal); Methodology (equal); Project administration (equal); Software (equal); Writing – original draft (equal); Writing – review & editing (equal). **Wibe A. de Jong**: Funding acquisition (supporting); Project administration (equal); Writing – original draft (equal); Writing – review & editing (equal).

DATA AVAILABILITY

The data that support the findings of this study are available within the article and its supplementary material.

APPENDIX: THREE-CENTER COULOMB POTENTIAL INTEGRALS

In this section, we present our approach for direct evaluation of contributions arising from the 3c-CP integrals to the sn-K intermediate **G** [Eq. (38)]. As opposed to the McMurchie–Davidson approach taken for the DF-*J*-engine in Sec. II B, we have adopted the Obara–Saika GTO integral recursions[98,140] for the evaluation of **G**. We refer the reader to Sec. II B for notation regarding basis functions and integrals used in this section.

For a particular shell pair $\phi_{\mathbf{a}}, \phi_{\mathbf{b}} \in \mathcal{B}$, and grid point $\mathbf{r}_i \in \mathcal{Q}$, we define the auxiliary integrals, $\Theta^n_{\mathbf{a}}$ and $\Xi_{\mathbf{ab}}$ where

$$\Theta^n_{\mathbf{a}+1_q} = (\mathbf{R}_p - \mathbf{R}_{\mathbf{a}})_q \Theta^n_{\mathbf{a}} - (\mathbf{P} - \mathbf{r}_i)_q \Theta^{n+1}_{\mathbf{a}} + \frac{a_q}{2(\zeta_{\mathbf{a}} + \zeta_{\mathbf{b}})} \left(\Theta^n_{\mathbf{a}-1_q} - \Theta^{n+1}_{\mathbf{a}-1_q} \right), \tag{A1}$$

$$\Xi_{\mathbf{a}(\mathbf{b}+1_q)} = \Xi_{(\mathbf{a}+1_q)\mathbf{b}} + (\mathbf{R}_{\mathbf{a}} - \mathbf{R}_{\mathbf{b}})_q \Xi_{\mathbf{ab}}, \tag{A2}$$

and

$$\Theta^n_0 = F_n((\zeta_{\mathbf{a}} + \zeta_{\mathbf{b}})|\mathbf{R}_p - \mathbf{r}_i|^2), \qquad \Xi_{\mathbf{a}0} = \Theta^0_{\mathbf{a}}, \qquad A_{\mathbf{ab}i} \equiv \Xi_{\mathbf{ab}}. \tag{A3}$$

F_n is the Boys function defined in Eq. (22). Will refer to Eqs. (A1) and (A2) as the vertical (VRR) and horizontal (HRR) OS recursions, respectively, in the following. In the canonical two-step OS algorithm, VRR intermediates are constructed and assembled into target integrals via the HRR. For brevity, the following discussion considers the case that $\phi_{\mathbf{a}}$ and $\phi_{\mathbf{b}}$ are primitive GTO functions. This may be generalized to contracted functions by contracting the Θ intermediates with basis coefficients between the VRR and HRR.[140] There are two important aspects of these expressions for the 3c-CP:

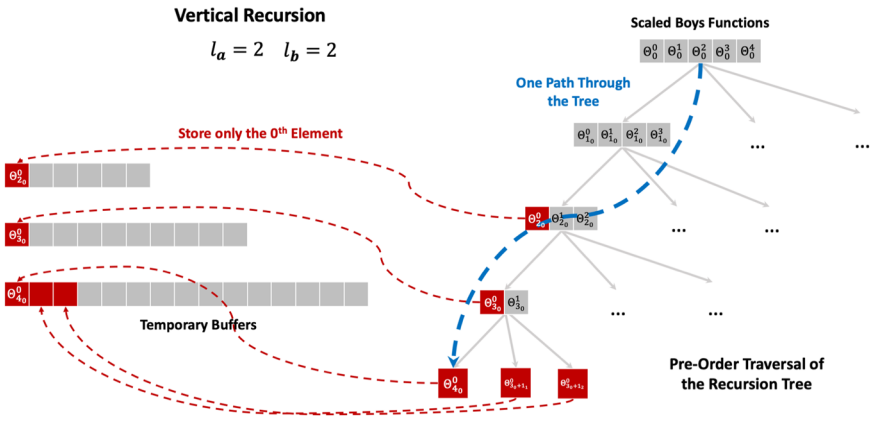

FIG. 8. A pre-order tree transversal approach for the evaluation of the VRR expressed in Eq. (A1). The nodes and edges of this tree represent intermediate integrals and their VRR connections, respectively. Gray tiles represent temporary terms that must be computed to form the Θ values (red) required for the HRR. Only one path through the VRR tree transversal is shown for brevity.

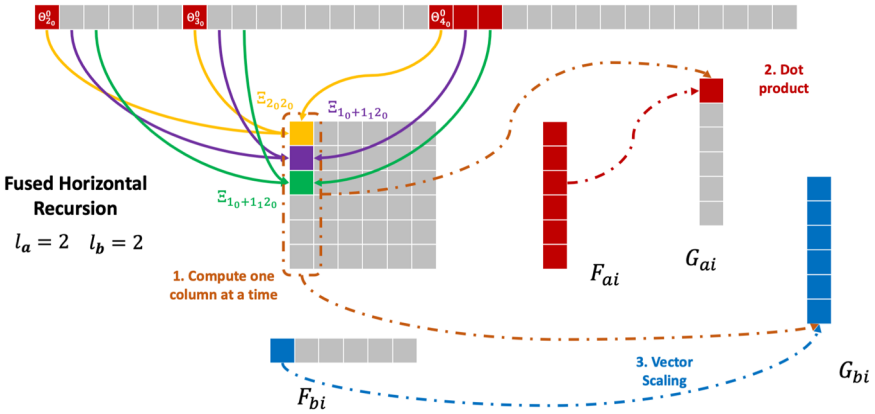

FIG. 9. onstruction of the $\Xi_{a(b)}$ column by column and the subsequent merger with the contraction with the elements of $F_{a/bi}$ to form **G**. The full 3c-CP integral tensor is never fully materialized.

1. In contrast to their application to the evaluation of ERIs,[140] only the Θ^0 intermediates are required for the assembly of target integrals via the HRR. As such, for a particular **a** and **b**, the VRR recursion requires the evaluation of many intermediate quantities that do not contribute to the HRR.
2. Both the HRR and VRR are independent in the grid point and are thus well suited to be parallelized in this dimension. On GPU architectures, each thread can independently perform the same VRR and HRR steps for different grid points (batch level parallelism) with minimal thread divergence. However, adopting this simple approach can be resource (e.g., register and shared memory) intensive, which can hinder the number of thread blocks that can be executed concurrently on a particular SM.

In the following, we develop an efficient algorithm for the transversal of the VRR and HRR recursions for the sn-K method to minimize GPU resource requirements. Note that both the VRR and HRR computations use double-precision (DP) floating point arithmetic. In addition, all the values used in the computation are stored as DP values in registers or in memory (local, shared, or global).

1. VRR implementation

Figure 8 depicts our approach for implementing the VRR in the context of the sn-K method. First, we cast the recursion as a tree traversal, where the nodes of the tree represent the intermediate integrals and the edges between nodes represent the direct connections between the integrals via the VRR. The levels of the tree represent Θ^n intermediates with the same total angular momentum, L, with the root of the tree representing the state $L = 0$. For a target **a** and **b**, $L \in [0, l_\mathbf{a} + l_\mathbf{b} + 1]$, storage is kept to a minimum by performing a pre-order transversal of the VRR tree. This allows for the direct

storage of Θ^0 intermediates without having to form all elements of a tree level simultaneously. For example, via the pre-order transversal depicted in Fig. 8, only 15 Θ values need to be held simultaneously to evaluate every target integral for $l_\mathbf{a} = l_\mathbf{b} = 2$ as compared to 20 Θ values for a breath first search traversal. The 15 DP values require 15 DP registers or 30 single-precision (SP) registers as reported by NVIDIA's `nvcc` compiler (one DP register is equivalent to two SP registers). Additionally, 30 DP intermediate values must be stored in shared memory, values that are needed for the HRR step. As such, our approach of the VRR step requires $(l_\mathbf{a} + l_\mathbf{b} + 1)(l_\mathbf{a} + l_\mathbf{b} + 2)/2$ DP registers, and $\sum_{i=1}^{l_\mathbf{b}} (l_\mathbf{a} + i + 1)(l_\mathbf{a} + i + 2)/2$ DP intermediate values in the GPU shared memory (multiply each quantity by 2 to report them as SP values). The quantities must then be multiplied with the total number of threads within an execution block to get the total resources needed. For small angular momenta (e.g., $l_\mathbf{a/b} < 2$), we do not utilize shared memory and keep all values in registers, including the intermediate values. This has shown to provide the best performance at the expense of an increased register count.

2. Merged HRR implementation

Given the required Θ^0 intermediates from the VRR, Fig. 9 depicts our approach for merging the HRR with the contraction with elements of $F_{\mathbf{a/b}i}$ to directly form its contributions to $G_{\mathbf{a/b}i}$ in Eq. (38). In this approach, the full 3c-CP integral tensor (which quadratically grows with angular momenta) need not be materialized in order to perform the contraction in Eq. (38). For example, for $l_\mathbf{a} = l_\mathbf{b} = 2$, the full 6×6 integral tensor occupies 36 DP registers, which would constitute a significant bottleneck for resource utilization on modern GPU hardware, particularly when vectorizing over many grid points. In our approach, we only need to materialize a single column of the target integral at a time, which in the preceding example only requires 6 DP registers (a 6-fold reduction). Additionally, we need 12 DP registers to store the values from $F_{\mathbf{a/b}i}$ and the results for $G_{\mathbf{a/b}i}$. The values for $F_{\mathbf{a/b}i}$ can be preloaded to hide the latency of reading data from the GPU main memory. In general, for the HRR step our approach needs $(l_\mathbf{a} + 1)(l_\mathbf{a} + 2) + (l_\mathbf{b} + 1)(l_\mathbf{a} + 2)/2$ DP registers (multiply the quantity by 2 to report used SP registers as outlined by the NVIDIA tools). This value must be multiplied with the total number of threads within an thread block to obtain the final resource utilization.

For the GPU implementation, we have constructed device kernels for different angular momenta. More specifically, we have constructed an in-house code generator that performs the pre-order traversal and fuses the horizontal recursion with the other contractions and generates CUDA code. The current implementation exploits the independent execution across the grid points, each GPU thread executing the same VRR and HRR but on different grid points. For small angular momenta, we do not utilize shared memory and keep all intermediate values in registers. For larger values, we make use of shared memory, which is dynamically allocated given the angular momenta. Therefore, the main strategy, given a shell pair of angular momenta $l_\mathbf{a}$ and $l_\mathbf{b}$, is to first group the grid points for that specific shell pair, launch a specialized kernel on the GPU and perform the batched VRR and HRR computations, and finally obtain the contracted $G_{\mathbf{a/b}i}$ stored in device memory.

REFERENCES

[1] C. L. Janssen and I. M. Nielsen, *Parallel Computing in Quantum Chemistry* (CRC Press, 2008).
[2] W. A. De Jong, E. Bylaska, N. Govind, C. L. Janssen, K. Kowalski, T. Müller, I. M. B. Nielsen, H. J. J. van Dam, V. Veryazov, and R. Lindh, "Utilizing high performance computing for chemistry: Parallel computational chemistry," Phys. Chem. Chem. Phys. **12**, 6896–6920 (2010).
[3] J. A. Calvin, C. Peng, V. Rishi, A. Kumar, and E. F. Valeev, "Many-body quantum chemistry on massively parallel computers," Chem. Rev. **121**, 1203–1231 (2021).
[4] V. Gavini, S. Baroni, V. Blum, D. R. Bowler, A. Buccheri, J. R. Chelikowsky, S. Das, W. Dawson, P. Delugas, M. Dogan *et al.*, "Roadmap on electronic structure codes in the exascale era," arXiv:2209.12747 (2022).
[5] M. S. Gordon, G. Barca, S. S. Leang, D. Poole, A. P. Rendell, J. L. Galvez Vallejo, and B. Westheimer, "Novel computer architectures and quantum chemistry," J. Phys. Chem. A **124**, 4557–4582 (2020).
[6] M. S. Gordon and T. L. Windus, "Editorial: Modern architectures and their impact on electronic structure theory," Chem. Rev. **120**, 9015–9020 (2020).
[7] K. Yasuda, "Accelerating density functional calculations with graphics processing unit," J. Chem. Theory Comput. **4**, 1230–1236 (2008).
[8] J. Kalinowski, F. Wennmohs, and F. Neese, "Arbitrary angular momentum electron repulsion integrals with graphical processing units: Application to the resolution of identity Hartree–Fock method," J. Chem. Theory Comput. **13**, 3160–3170 (2017).
[9] J. Kussmann and C. Ochsenfeld, "Employing openCL to accelerate ab initio calculations on graphics processing units," J. Chem. Theory Comput. **13**, 2712–2716 (2017).
[10] H. Laqua, T. H. Thompson, J. Kussmann, and C. Ochsenfeld, "Highly efficient, linear-scaling seminumerical exact-exchange method for graphic processing units," J. Chem. Theory Comput. **16**, 1456–1468 (2020).
[11] J. Kussmann and C. Ochsenfeld, "Preselective screening for linear-scaling exact exchange-gradient calculations for graphics processing units and general strong-scaling massively parallel calculations," J. Chem. Theory Comput. **11**, 918–922 (2015).
[12] J. Kussmann and C. Ochsenfeld, "Hybrid CPU/GPU integral engine for strong-scaling *ab initio* methods," J. Chem. Theory Comput. **13**, 3153–3159 (2017).
[13] J. Kussmann, H. Laqua, and C. Ochsenfeld, "Highly efficient resolution-of-identity density functional theory calculations on central and graphics processing units," J. Chem. Theory Comput. **17**, 1512–1521 (2021).
[14] H. Laqua, J. C. B. Dietschreit, J. Kussmann, and C. Ochsenfeld, "Accelerating hybrid density functional theory molecular dynamics simulations by seminumerical integration, resolution-of-the-identity approximation, and graphics processing units," J. Chem. Theory Comput. **18**, 6010–6020 (2022).
[15] I. S. Ufimtsev and T. J. Martinez, "Quantum chemistry on graphical processing units. 2. Direct self-consistent-field implementation," J. Chem. Theory Comput. **5**, 1004–1015 (2009).
[16] N. Luehr, A. Sisto, and T. J. Martínez, "Gaussian basis set Hartree–Fock, density functional theory, and beyond on GPUs," in *Electronic Structure Calculations on Graphics Processing Units* (John Wiley and Sons, Ltd, 2016), Chap. 4, pp. 67–100.
[17] D. B. Williams-Young, W. A. de Jong, H. J. J. van Dam, and C. Yang, "On the efficient evaluation of the exchange correlation potential on graphics processing unit clusters," Front. Chem. **8**, 581058 (2020).
[18] D. B. Williams-Young, A. Bagusetty, W. A. de Jong, D. Doerfler, H. J. J. van Dam, Á. Vázquez-Mayagoitia, T. L. Windus, and C. Yang, "Achieving performance portability in Gaussian basis set density functional theory on accelerator based architectures in NWChemEx," Parallel Comput. **108**, 102829 (2021).
[19] H. Ahmed, D. B. Williams-Young, K. Z. Ibrahim, and C. Yang, "Performance modeling and tuning for DFT calculations on heterogeneous architectures," in *2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)* (IEEE, 2021), pp. 714–722.
[20] A. Asadchev and M. S. Gordon, "New multithreaded hybrid CPU/GPU approach to Hartree–Fock," J. Chem. Theory Comput. **8**, 4166–4176 (2012).
[21] G. M. J. Barca, J. L. Galvez-Vallejo, D. L. Poole, A. P. Rendell, and M. S. Gordon, "High-performance, graphics processing unit-accelerated Fock build algorithm," J. Chem. Theory Comput. **16**, 7232–7238 (2020).

[22]G. M. J. Barca, D. L. Poole, J. L. G. Vallejo, M. Alkan, C. Bertoni, A. P. Rendell, and M. S. Gordon, "Scaling the Hartree-Fock matrix build on summit," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis* (IEEE, 2020), pp. 1–14.
[23]G. M. J. Barca, M. Alkan, J. L. Galvez-Vallejo, D. L. Poole, A. P. Rendell, and M. S. Gordon, "Faster self-consistent field (SCF) calculations on GPU clusters," J. Chem. Theory Comput. **17**, 7486–7503 (2021).
[24]M. Manathunga, C. Jin, V. W. D. Cruzeiro, Y. Miao, D. Mu, K. Arumugam, K. Keipert, H. M. Aktulga, K. M. Merz, Jr., and A. W. Götz, "Harnessing the power of multi-GPU acceleration into the quantum interaction computational kernel program," J. Chem. Theory Comput. **17**, 3955–3966 (2021).
[25]M. Manathunga, Y. Miao, D. Mu, A. W. Götz, and K. M. Merz, Jr., "Parallel implementation of density functional theory methods in the quantum interaction computational kernel program," J. Chem. Theory Comput. **16**, 4315–4326 (2020).
[26]M. Manathunga, H. M. Aktulga, A. W. Götz, and K. M. Merz, Jr., "Quantum mechanics/molecular mechanics simulations on NVIDIA and AMD graphics processing units," J. Chem. Inf. Model. **63**, 711 (2023).
[27]S. Maintz, B. Eck, and R. Dronskowski, "Speeding up plane-wave electronic-structure calculations using graphics-processing units," Comput. Phys. Commun. **182**, 1421–1427 (2011).
[28]L. Wang, Y. Wu, W. Jia, W. Gao, X. Chi, and L.-W. Wang, "Large scale plane wave pseudopotential density functional theory calculations on GPU clusters," in *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis* (IEEE, 2011), pp. 1–10.
[29]M. Hacene, A. Anciaux-Sedrakian, X. Rozanska, D. Klahr, T. Guignon, and P. Fleurat-Lessard, "Accelerating VASP electronic structure calculations using graphic processing units," J. Comput. Chem. **33**, 2581–2589 (2012).
[30]P. Giannozzi, O. Baseggio, P. Bonfà, D. Brunato, R. Car, I. Carnimeo, C. Cavazzoni, S. De Gironcoli, P. Delugas, F. Ferrari Ruffino *et al.*, "Quantum ESPRESSO toward the exascale," J. Chem. Phys. **152**, 154105 (2020).
[31]E. Aprà, E. J. Bylaska, W. A. de Jong, N. Govind, K. Kowalski, T. P. Straatsma, M. Valiev, H. J. J. van Dam, Y. Alexeev, J. Anchell, V. Anisimov, F. W. Aquino, R. Atta-Fynn, J. Autschbach, N. P. Bauman, J. C. Becca, D. E. Bernholdt, K. Bhaskaran-Nair, S. Bogatko, P. Borowski, J. Boschen, J. Brabec, A. Bruner, E. Cauët, Y. Chen, G. N. Chuev, C. J. Cramer, J. Daily, M. J. O. Deegan, T. H. Dunning, M. Dupuis, K. G. Dyall, G. I. Fann, S. A. Fischer, A. Fonari, H. Früchtl, L. Gagliardi, J. Garza, N. Gawande, S. Ghosh, K. Glaesemann, A. W. Götz, J. Hammond, V. Helms, E. D. Hermes, K. Hirao, S. Hirata, M. Jacquelin, L. Jensen, B. G. Johnson, H. Jónsson, R. A. Kendall, M. Klemm, R. Kobayashi, V. Konkov, S. Krishnamoorthy, M. Krishnan, Z. Lin, R. D. Lins, R. J. Littlefield, A. J. Logsdail, K. Lopata, W. Ma, A. V. Marenich, J. Martin del Campo, D. Mejia-Rodriguez, J. E. Moore, J. M. Mullin, T. Nakajima, D. R. Nascimento, J. A. Nichols, P. J. Nichols, J. Nieplocha, A. Otero-de-la-Roza, B. Palmer, A. Panyala, T. Pirojsirikul, B. Peng, R. Peverati, J. Pittner, L. Pollack, R. M. Richard, P. Sadayappan, G. C. Schatz, W. A. Shelton, D. W. Silverstein, D. M. A. Smith, T. A. Soares, D. Song, M. Swart, H. L. Taylor, G. S. Thomas, V. Tipparaju, D. G. Truhlar, K. Tsemekhman, T. Van Voorhis, Á. Vázquez-Mayagoitia, P. Verma, O. Villa, A. Vishnu, K. D. Vogiatzis, D. Wang, J. H. Weare, M. J. Williamson, T. L. Windus, K. Woliński, A. T. Wong, Q. Wu, C. Yang, Q. Yu, M. Zacharias, Z. Zhang, Y. Zhao, and R. J. Harrison, "NWChem: Past, present, and future," J. Chem. Phys. **152**, 184102 (2020).
[32]K. Wilkinson and C.-K. Skylaris, "Porting ONETEP to graphical processing unit-based coprocessors. 1. FFT box operations," J. Comput. Chem. **34**, 2446–2459 (2013).
[33]X. Andrade and A. Aspuru-Guzik, "Real-space density functional theory on graphical processing units: Computational approach and comparison to Gaussian basis set methods," J. Chem. Theory Comput. **9**, 4360–4373 (2013).
[34]S. Hakala, V. Havu, J. Enkovaara, and R. Nieminen, "Parallel electronic structure calculations using multiple graphics processing units (GPUs)," in *Applied Parallel and Scientific Computing*, edited by P. Manninen and P. Öster (Springer, Berlin, Heidelberg, 2013), pp. 63–76.
[35]S. Das, P. Motamarri, V. Subramanian, D. M. Rogers, and V. Gavini, "DFT-FE 1.0: A massively parallel hybrid CPU-GPU density functional theory code using finite-element discretization," Comput. Phys. Commun. **280**, 108473 (2022).
[36]L. Genovese, M. Ospici, T. Deutsch, J.-F. Méhaut, A. Neelov, and S. Goedecker, "Density functional theory calculation on many-cores hybrid central processing unit-graphic processing unit architectures," J. Chem. Phys. **131**, 034103 (2009).
[37]H. van Schoot and L. Visscher, "GPU acceleration for density functional theory with slater-type orbitals," in *Electronic Structure Calculations on Graphics Processing Units* (John Wiley and Sons, Ltd, 2016), Chap. 5, pp. 101–114.
[38]T. Yoshikawa, N. Komoto, Y. Nishimura, and H. Nakai, "GPU-accelerated large-scale excited-state simulation based on divide-and-conquer time-dependent density-functional tight-binding," J. Comput. Chem. **40**, 2778–2786 (2019).
[39]W. P. Huhn, B. Lange, V. W.-z. Yu, M. Yoon, and V. Blum, "GPU acceleration of all-electron electronic structure theory using localized numeric atom-centered basis functions," Comput. Phys. Commun. **254**, 107314 (2020).
[40]A. E. DePrince III and J. R. Hammond, "Coupled cluster theory on graphics processing units I. The coupled cluster doubles method," J. Chem. Theory Comput. **7**, 1287–1295 (2011).
[41]A. E. DePrince III, J. R. Hammond, and C. D. Sherrill, "Iterative coupled-cluster methods on graphics processing units," in *Electronic Structure Calculations on Graphics Processing Units* (John Wiley & Sons, Ltd, 2016), Chap. 13, pp. 279–300.
[42]A. Asadchev and M. S. Gordon, "Fast and flexible coupled cluster implementation," J. Chem. Theory Comput. **9**, 3385–3392 (2013).
[43]I. A. Kaliman and A. I. Krylov, "New algorithm for tensor contractions on multi-core CPUs, GPUs, and accelerators enables CCSD and EOM-CCSD calculations with over 1000 basis functions on a single compute node," J. Comput. Chem. **38**, 842–853 (2017).
[44]B. S. Fales, E. R. Curtis, K. G. Johnson, D. Lahana, S. Seritan, Y. Wang, H. Weir, T. J. Martínez, and E. G. Hohenstein, "Performance of coupled-cluster singles and doubles on modern stream processing architectures," J. Chem. Theory Comput. **16**, 4021–4028 (2020).
[45]E. G. Hohenstein and T. J. Martínez, "GPU acceleration of rank-reduced coupled-cluster singles and doubles," J. Chem. Phys. **155**, 184110 (2021).
[46]W. Ma, S. Krishnamoorthy, O. Villay, and K. Kowalski, "Acceleration of streamed tensor contraction expressions on GPGPU-based clusters," in *2010 IEEE International Conference on Cluster Computing* (IEEE, 2010), pp. 207–216.
[47]W. Ma, K. Bhaskaran-Nair, O. Villa, E. Aprà, A. Tumeo, S. Krishnamoorthy, and K. Kowalski, "Perturbative coupled-cluster methods on graphics processing units: Single- and multi-reference formulations," in *Electronic Structure Calculations on Graphics Processing Units* (John Wiley & Sons, Ltd, 2016), Chap. 14, pp. 301–326.
[48]C. Peng, J. A. Calvin, and E. F. Valeev, "Coupled-cluster singles, doubles and perturbative triples with density fitting approximation for massively parallel heterogeneous platforms," Int. J. Quantum Chem. **119**, e25894 (2019).
[49]J. V. Pototschnig, A. Papadopoulos, D. I. Lyakh, M. Repisky, L. Halbert, A. Severo Pereira Gomes, H. J. A. Jensen, and L. Visscher, "Implementation of relativistic coupled cluster theory for massively parallel GPU-accelerated computing architectures," J. Chem. Theory Comput. **17**, 5509–5529 (2021).
[50]R. Olivares-Amaya, M. A. Watson, R. G. Edgar, L. Vogt, Y. Shao, and A. Aspuru-Guzik, "Accelerating correlated quantum chemistry calculations using graphical processing units and a mixed precision matrix multiplication library," J. Chem. Theory Comput. **6**, 135–144 (2010).
[51]R. Olivares-Amaya, A. Jinich, M. A. Watson, and A. Aspuru-Guzik, "GPU acceleration of second-order Møller–Plesset perturbation theory with resolution of identity," in *Electronic Structure Calculations on Graphics Processing Units* (John Wiley & Sons, Ltd, 2016), Chap. 12, pp. 259–278.
[52]C. Song and T. J. Martínez, "Atomic orbital-based SOS-MP2 with tensor hypercontraction. I. GPU-based tensor construction and exploiting sparsity," J. Chem. Phys. **144**, 174111 (2016).
[53]S. A. Maurer, J. Kussmann, and C. Ochsenfeld, "Communication: A reduced scaling J-engine based reformulation of SOS-MP2 using graphics processing units," J. Chem. Phys. **141**, 051106 (2014).
[54]D. Bykov and T. Kjaergaard, "The GPU-enabled divide-expand-consolidate RI-MP2 method (DEC-RI-MP2)," J. Comput. Chem. **38**, 228–237 (2017).
[55]G. M. J. Barca, S. C. McKenzie, N. J. Bloomfield, A. T. B. Gilbert, and P. M. W. Gill, "Q-MP2-OS: Møller–Plesset correlation energy by quadrature," J. Chem. Theory Comput. **16**, 1568–1577 (2020).
[56]G. M. Barca, J. L. G. Vallejo, D. L. Poole, M. Alkan, R. Stocks, A. P. Rendell, and M. S. Gordon, "Enabling large-scale correlated electronic structure calculations: Scaling the RI-MP2 method on summit," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Association for Computing Machinery, New York, 2021), pp. 1–15.

[57]E. G. Hohenstein, N. Luehr, I. S. Ufimtsev, and T. J. Martínez, "An atomic orbital-based formulation of the complete active space self-consistent field method on graphical processing units," J. Chem. Phys. **142**, 224103 (2015).
[58]B. S. Fales and T. J. Martínez, "Efficient treatment of large active spaces through multi-GPU parallel implementation of direct configuration interaction," J. Chem. Theory Comput. **16**, 1586–1596 (2020).
[59]J. W. Mullinax, E. Maradzike, L. N. Koulias, M. Mostafanejad, E. Epifanovsky, G. Gidofalvi, and A. E. DePrince III, "Heterogeneous CPU + GPU algorithm for variational two-electron reduced-density matrix-driven complete active-space self-consistent field theory," J. Chem. Theory Comput. **15**, 6164–6178 (2019).
[60]T. P. Straatsma, R. Broer, S. Faraji, R. W. A. Havenith, L. E. A. Suarez, R. K. Kathir, M. Wibowo, and C. de Graaf, "GronOR: Massively parallel and GPU-accelerated non-orthogonal configuration interaction for large molecular systems," J. Chem. Phys. **152**, 064111 (2020).
[61]P. Maris, C. Yang, D. Oryspayev, and B. Cook, "Accelerating an iterative eigensolver for nuclear structure configuration interaction calculations on GPUs using OpenACC," J. Comput. Sci. **59**, 101554 (2022).
[62]D. Kothe, S. Lee, and I. Qualters, "Exascale computing in the United States," Comput. Sci. Eng. **21**, 17–29 (2019).
[63]F. Alexander, A. Almgren, J. Bell, A. Bhattacharjee, J. Chen, P. Colella, D. Daniel, J. DeSlippe, L. Diachin, E. Draeger, A. Dubey, T. Dunning, T. Evans, I. Foster, M. Francois, T. Germann, M. Gordon, S. Habib, M. Halappanavar, S. Hamilton, W. Hart, Z. Huang, A. Hungerford, D. Kasen, P. R. C. Kent, T. Kolev, D. B. Kothe, A. Kronfeld, Y. Luo, P. Mackenzie, D. McCallen, B. Messer, S. Mniszewski, C. Oehmen, A. Perazzo, D. Perez, D. Richards, W. J. Rider, R. Rieben, K. Roche, A. Siegel, M. Sprague, C. Steefel, R. Stevens, M. Syamlal, M. Taylor, J. Turner, J.-L. Vay, A. F. Voter, T. L. Windus, and K. Yelick, "Exascale applications: Skin in the game," Philos. Trans. R. Soc. London, Ser. A **378**, 20190056 (2020).
[64]S. Ashby, P. Beckman, J. Chen, P. Colella, B. Collins, D. Crawford, J. Dongarra, D. Kothe, R. Lusk, P. Messina *et al.*, "The opportunities and challenges of exascale computing," Summary Report of the Advanced Scientific Computing Advisory Committee (ASCAC) Subcommittee (2010), pp. 1–77.
[65]S. Amarasinghe, M. Hall, R. Lethin, K. Pingali, D. Quinlan, V. Sarkar, J. Shalf, R. Lucas, K. Yelick, P. Balanji *et al.*, "Exascale programming challenges," in *Proceedings of the Workshop on Exascale Programming Challenges* (US Department of Energy, Office of Science, Office of Advanced Scientific Computing Research (ASCR), Marina del Rey, CA, 2011).
[66]J. A. Calvin, C. A. Lewis, and E. F. Valeev, "Scalable task-based algorithm for multiplication of block-rank-sparse matrices," in *Proceedings of the 5th Workshop on Irregular Applications: Architectures and Algorithms* (Association for Computing Machinery, New York, 2015), pp. 1–8.
[67]J. A. Calvin and E. F. Valeev, "Task-based algorithm for matrix multiplication: A step towards block-sparse tensor computing," arXiv:1504.05046 (2015).
[68]D. I. Lyakh, "An efficient tensor transpose algorithm for multicore CPU, Intel Xeon Phi, and NVidia Tesla GPU," Comput. Phys. Commun. **189**, 84–91 (2015).
[69]J. Kim, A. Sukumaran-Rajam, V. Thumma, S. Krishnamoorthy, A. Panyala, L.-N. Pouchet, A. Rountev, and P. Sadayappan, "A code generator for high-performance tensor contractions on GPUs," in *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)* (IEEE, 2019), pp. 85–95.
[70]D. T. Popovici, A. Canning, Z. Zhao, L.-W. Wang, and J. Shalf, "A systematic approach to improving data locality across Fourier transforms and linear algebra operations," in *Proceedings of the ACM International Conference on Supercomputing* (Association for Computing Machinery, New York, 2021), pp. 329–341.
[71]A. Ayala, S. Tomov, A. Haidar, and J. Dongarra, "heFFTe: Highly efficient FFT for exascale," in *Computational Science–ICCS 2020: 20th International Conference, Amsterdam, The Netherlands, June 3–5, 2020, Proceedings, Part I* (Springer, 2020), pp. 262–275.
[72]F. Franchetti, D. G. Spampinato, A. Kulkarni, D. T. Popovici, T. M. Low, M. Franusich, A. Canning, P. McCorquodale, B. Van Straalen, and P. Colella, "FFTX and SpectralPack: A first look," in *2018 IEEE 25th International Conference on High Performance Computing Workshops (HiPCW)* (IEEE, 2018), pp. 18–27.
[73]N. Luehr, I. S. Ufimtsev, and T. J. Martínez, "Dynamic precision for electron repulsion integral evaluation on graphical processing units (GPUs)," J. Chem. Theory Comput. **7**, 949–954 (2011).
[74]A. Asadchev, V. Allada, J. Felder, B. M. Bode, M. S. Gordon, and T. L. Windus, "Uncontracted RYs quadrature implementation of up to G functions on graphical processing units," J. Chem. Theory Comput. **6**, 696–704 (2010).
[75]Y. Miao and K. M. Merz, Jr., "Acceleration of electron repulsion integral evaluation on graphics processing units via use of recurrence relations," J. Chem. Theory Comput. **9**, 965–976 (2013).
[76]Y. Miao and K. M. Merz, Jr., "Acceleration of high angular momentum electron repulsion integrals and integral derivatives on graphics processing units," J. Chem. Theory Comput. **11**, 1449–1462 (2015).
[77]G. Shi, V. Kindratenko, I. Ufimtsev, and T. Martinez, "Direct self-consistent field computations on GPU clusters," in *2010 IEEE International Symposium on Parallel & Distributed Processing (IPDPS)* (IEEE, 2010), pp. 1–8.
[78]K. G. Johnson, S. Mirchandaney, E. Hoag, A. Heirich, A. Aiken, and T. J. Martínez, "Multinode multi-GPU two-electron integrals: Code generation using the regent language," J. Chem. Theory Comput. **18**, 6522–6536 (2022).
[79]L. Greengard and V. Rokhlin, "A fast algorithm for particle simulations," J. Comput. Phys. **73**, 325–348 (1987).
[80]C. A. White, B. G. Johnson, P. M. W. Gill, and M. Head-Gordon, "The continuous fast multipole method," Chem. Phys. Lett. **230**, 8–16 (1994).
[81]G. Reza Ahmadi and J. Almlöf, "The Coulomb operator in a Gaussian product basis," Chem. Phys. Lett. **246**, 364–370 (1995).
[82]C. A. White and M. Head-Gordon, "A *J* matrix engine for density functional theory calculations," J. Chem. Phys. **104**, 2620–2629 (1996).
[83]T. R. Adams, R. D. Adamson, and P. M. W. Gill, "A tensor approach to two-electron matrix elements," J. Chem. Phys. **107**, 124–131 (1997).
[84]Y. Shao and M. Head-Gordon, "An improved *J* matrix engine for density functional theory calculations," Chem. Phys. Lett. **323**, 425–433 (2000).
[85]J. L. Whitten, "Coulombic potential energy integrals and approximations," J. Chem. Phys. **58**, 4496–4501 (1973).
[86]O. Vahtras, J. Almlöf, and M. W. Feyereisen, "Integral approximations for LCAO-SCF calculations," Chem. Phys. Lett. **213**, 514–518 (1993).
[87]R. A. Friesner, "Solution of self-consistent field electronic structure equations by a pseudospectral method," Chem. Phys. Lett. **116**, 39–43 (1985).
[88]M. N. Ringnalda, M. Belhadj, and R. A. Friesner, "Pseudospectral Hartree–Fock theory: Applications and algorithmic improvements," J. Chem. Phys. **93**, 3397–3407 (1990).
[89]F. Neese, F. Wennmohs, A. Hansen, and U. Becker, "Efficient, approximate and parallel Hartree–Fock and hybrid DFT calculations. A 'chain-of-spheres' algorithm for the Hartree–Fock exchange," Chem. Phys. **356**, 98–109 (2009).
[90]H. Laqua, J. Kussmann, and C. Ochsenfeld, "Accelerating seminumerical Fock-exchange calculations using mixed single- and double-precision arithmethic," J. Chem. Phys. **154**, 214116 (2021).
[91]A. Szabo and N. S. Ostlund, *Modern Quantum Chemistry: Introduction to Advanced Electronic Structure Theory* (Courier Corporation, 2012).
[92]R. G. Parr and Y. Weitao, *Density-Functional Theory of Atoms and Molecules* (Oxford University Press, 1995).
[93]R. Ahlrichs, "Efficient evaluation of three-center two-electron integrals over Gaussian functions," Phys. Chem. Chem. Phys. **6**, 5119 (2004).
[94]D. S. Hollman, H. F. Schaefer, and E. F. Valeev, "A tight distance-dependent estimator for screening three-center Coulomb integrals over Gaussian basis functions," J. Chem. Phys. **142**, 154106 (2015).
[95]E. F. Valeev and T. Shiozaki, "Comment on 'A tight distance-dependent estimator for screening three-center Coulomb integrals over Gaussian basis functions' [J. Chem. Phys. 142, 154106 (2015)]," J. Chem. Phys. **153**, 097101 (2020).
[96]A. Asadchev and E. F. Valeev, "Memory-efficient recursive evaluation of 3-center Gaussian integrals," J. Chem. Theory Comput. **19**, 1698 (2023).
[97]L. E. McMurchie and E. R. Davidson, "One- and two-electron integrals over Cartesian Gaussian functions," J. Comput. Phys. **26**, 218–231 (1978).
[98]S. Obara and A. Saika, "Efficient recursive computation of molecular integrals over Cartesian Gaussian functions," J. Chem. Phys. **84**, 3963–3974 (1986).
[99]See https://github.com/ValeevGroup/LibintX for LibintX.
[100]J. Calvin, E. Valeyev, C. Peng, D. Lewis, D. Williams-Young, K. Pierce, R. Richard, A. Asadchev, X. Wang, J. M. Waldrop, J. Zhang, M. Ambroise, and S. Slattery, Valeevgroup/tiledarray: 1.0.0, 2020.

[101]C. A. Lewis, J. A. Calvin, and E. F. Valeev, "Clustered low-rank tensor format: Introduction and application to fast construction of Hartree–Fock exchange," J. Chem. Theory Comput. **12**, 5868–5880 (2016).
[102]J. A. Pople, P. M. W. Gill, and B. G. Johnson, "Kohn—Sham density-functional theory within a finite basis set," Chem. Phys. Lett. **199**, 557–560 (1992).
[103]A. M. Burow and M. Sierka, "Linear scaling hierarchical integration scheme for the exchange-correlation term in molecular and periodic systems," J. Chem. Theory Comput. **7**, 3097–3104 (2011).
[104]A. Petrone, D. B. Williams-Young, S. Sun, T. F. Stetina, and X. Li, "An efficient implementation of two-component relativistic density functional theory with torque-free auxiliary variables," Eur. Phys. J. B **91**, 169 (2018).
[105]A. D. Becke, "A multicenter numerical integration scheme for polyatomic molecules," J. Chem. Phys. **88**, 2547–2553 (1988).
[106]R. E. Stratmann, G. E. Scuseria, and M. J. Frisch, "Achieving linear scaling in exchange-correlation density functional quadratures," Chem. Phys. Lett. **257**, 213–223 (1996).
[107]H. Laqua, J. Kussmann, and C. Ochsenfeld, "An improved molecular partitioning scheme for numerical quadratures in density functional theory," J. Chem. Phys. **149**, 204111 (2018).
[108]M. E. Mura and P. J. Knowles, "Improved radial grids for quadrature in molecular density-functional calculations," J. Chem. Phys. **104**, 9848–9858 (1996).
[109]C. W. Murray, N. C. Handy, and G. J. Laming, "Quadrature schemes for integrals of density functional theory," Mol. Phys. **78**, 997–1014 (1993).
[110]O. Treutler and R. Ahlrichs, "Efficient molecular numerical integration schemes," J. Chem. Phys. **102**, 346–354 (1995).
[111]P. M. W. Gill and S.-H. Chien, "Radial quadrature for multiexponential integrands," J. Comput. Chem. **24**, 732–740 (2003).
[112]S.-H. Chien and P. M. W. Gill, "SG-0: A small standard grid for DFT quadrature on large systems," J. Comput. Chem. **27**, 730–739 (2006).
[113]P. M. W. Gill, B. G. Johnson, and J. A. Pople, "A standard grid for density functional calculations," Chem. Phys. Lett. **209**, 506–512 (1993).
[114]V. I. Lebedev, "Quadratures on a sphere," USSR Comput. Math. Math. Phys. **16**, 10–24 (1976).
[115]F. Egidi, S. Sun, J. J. Goings, G. Scalmani, M. J. Frisch, and X. Li, "Two-component noncollinear time-dependent spin density functional theory for excited state calculations," J. Chem. Theory Comput. **13**, 2591–2603 (2017).
[116]Evaluation of the density and its gradient actually proceeds as a batched-DOT as discussed in Sec. II E.
[117]C. Fonseca Guerra, J. G. Snijders, G. te Velde, and E. J. Baerends, "Towards an order-N DFT method," Theor. Chem. Acc. **99**, 391–403 (1998).
[118]G. te Velde, F. M. Bickelhaupt, E. J. Baerends, C. Fonseca Guerra, S. J. A. van Gisbergen, J. G. Snijders, and T. Ziegler, "Chemistry with ADF," J. Comput. Chem. **22**, 931–967 (2001).
[119]W. Kohn, "Analytic properties of Bloch waves and Wannier functions," Phys. Rev. **115**, 809–821 (1959).
[120]B. Helmich-Paris, B. de Souza, F. Neese, and R. Izsák, "An improved chain of spheres for exchange algorithm," J. Chem. Phys. **155**, 104109 (2021).
[121]T. H. Thompson and C. Ochsenfeld, "Integral partition bounds for fast and effective screening of general one-, two-, and many-electron integrals," J. Chem. Phys. **150**, 044101 (2019).
[122]P. Sanders, K. Mehlhorn, M. Dietzfelbinger, and R. Dementiev, *Sequential and Parallel Algorithms and Data Structures* (Springer, 2019).
[123]A. Haidar, T. Dong, P. Luszczek, S. Tomov, and J. Dongarra, "Batched matrix computations on hardware accelerators based on GPUs," Int. J. High Perform. Comput. Appl. **29**, 193–208 (2015).
[124]A. Abdelfattah, A. Haidar, S. Tomov, and J. Dongarra, "Performance, design, and autotuning of batched GEMM for GPUs," in *High Performance Computing*, edited by J. M. Kunkel, P. Balaji, and J. Dongarra (Springer International Publishing, 2016), pp. 21–38.
[125]See https://github.com/NVIDIA/nccl for NCCL.
[126]See https://github.com/wavefunction91/GauXC for GauXC.
[127]K. Kowalski, R. Bair, N. P. Bauman, J. S. Boschen, E. J. Bylaska, J. Daily, W. A. de Jong, T. Dunning, Jr., N. Govind, R. J. Harrison, M. Keçeli, K. Keipert, S. Krishnamoorthy, S. Kumar, E. Mutlu, B. Palmer, A. Panyala, B. Peng, R. M. Richard, T. P. Straatsma, P. Sushko, E. F. Valeev, M. Valiev, H. J. J. van Dam, J. M. Waldrop, D. B. Williams-Young, C. Yang, M. Zalewski, and T. L. Windus, "From NWChem to NWChemEx: Evolving with the computational chemistry landscape," Chem. Rev. **121**, 4962–4998 (2021).
[128]C. Peng, C. A. Lewis, X. Wang, M. C. Clement, K. Pierce, V. Rishi, F. Pavošević, S. Slattery, J. Zhang, N. Teke, A. Kumar, C. Masteran, A. Asadchev, J. A. Calvin, and E. F. Valeev, "Massively parallel quantum chemistry: A high-performance research platform for electronic structure," J. Chem. Phys. **153**, 044120 (2020).
[129]R. Ditchfield, W. J. Hehre, and J. A. Pople, "Self-consistent molecular-orbital methods. IX. An extended Gaussian-type basis for molecular-orbital studies of organic molecules," J. Chem. Phys. **54**, 724 (1971).
[130]M. M. Francl, W. J. Pietro, W. J. Hehre, J. S. Binkley, M. S. Gordon, D. J. DeFrees, and J. A. Pople, "Self-consistent molecular orbital methods. XXIII. A polarization-type basis set for second-row elements," J. Chem. Phys. **77**, 3654 (1982).
[131]M. S. Gordon, J. S. Binkley, J. A. Pople, W. J. Pietro, and W. J. Hehre, "Self-consistent molecular-orbital methods. 22. Small split-valence basis sets for second-row elements," J. Am. Chem. Soc. **104**, 2797 (1982).
[132]F. Weigend, M. Häser, H. Patzelt, and R. Ahlrichs, "RI-MP2: Optimized auxiliary basis sets and demonstration of efficiency," Chem. Phys. Lett. **294**, 143–152 (1998).
[133]C. Adamo and V. Barone, "Toward reliable density functional methods without adjustable parameters: The PBE0 model," J. Chem. Phys. **110**, 6158–6170 (1999).
[134]See https://github.com/wavefunction91/ExchCXX for ExchCXX.
[135]M. Challacombe, C. White, and M. Head-Gordon, "Periodic boundary conditions and the fast multipole method," J. Chem. Phys. **107**, 10131–10140 (1997).
[136]K. N. Kudin and G. E. Scuseria, "A fast multipole method for periodic systems with arbitrary unit cell geometries," Chem. Phys. Lett. **283**, 61–68 (1998).
[137]P. P. Ewald, "Die berechnung optischer und elektrostatischer gitterpotentiale," Ann. Phys. **369**, 253–287 (1921).
[138]A. K. Dutta, F. Neese, and R. Izsák, "Accelerating the coupled-cluster singles and doubles method using the chain-of-sphere approximation," Mol. Phys. **116**, 1428–1434 (2018).
[139]R. B. Murphy, M. D. Beachy, R. A. Friesner, and M. N. Ringnalda, "Pseudospectral localized Møller–Plesset methods: Theory and calculation of conformational energies," J. Chem. Phys. **103**, 1481–1490 (1995).
[140]P. M. W. Gill, M. Head-Gordon, and J. A. Pople, "An efficient algorithm for the generation of two-electron repulsion integrals over Gaussian basis functions," Int. J. Quantum Chem. **36**, 269–280 (1989).